

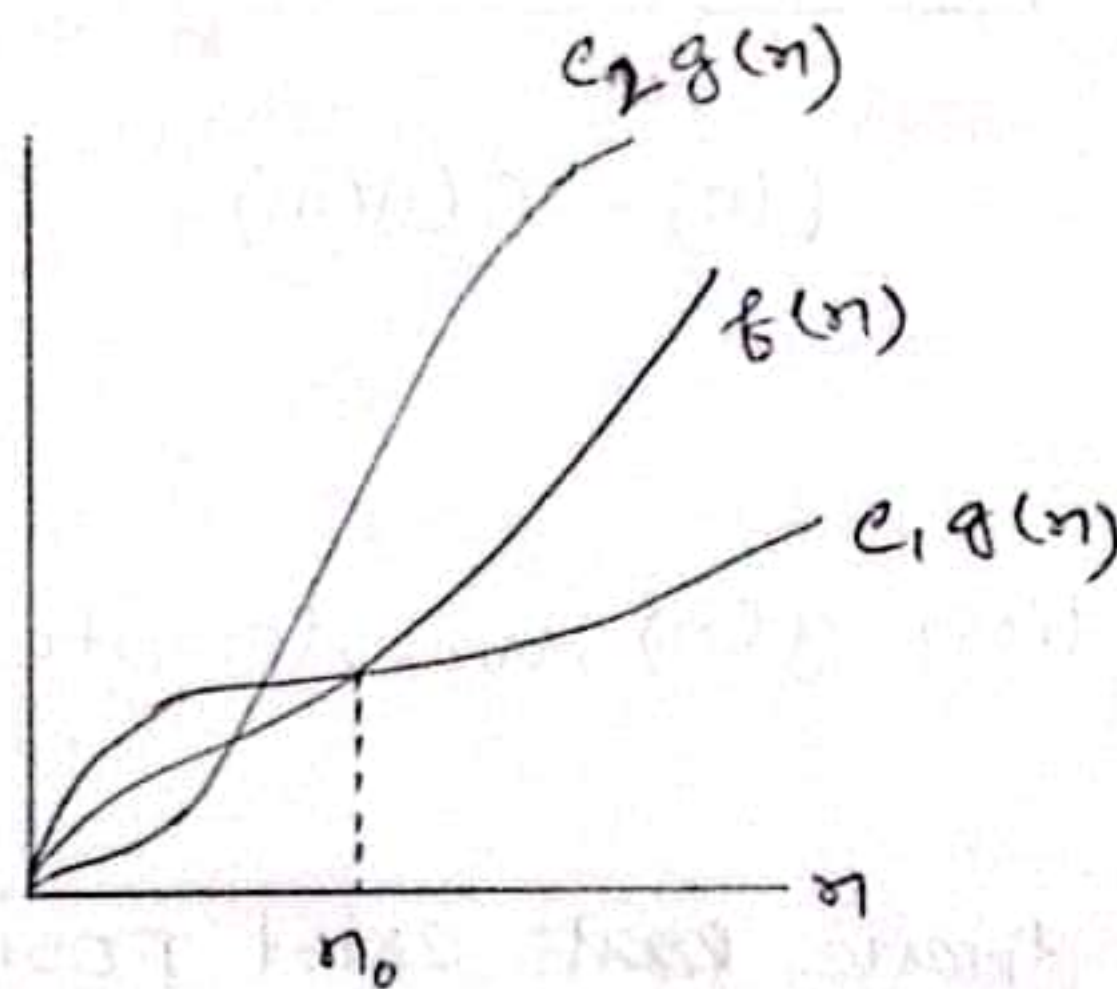
COMPLEXITY ANALYSIS

Θ Notation

For a given function $g(n)$, we denote by $\Theta(g(n))$ the set of functions

$$\Theta(g(n)) = \{ f(n) : \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0 \}$$

we say that $g(n)$ is an asymptotically tight bound for $f(n)$.



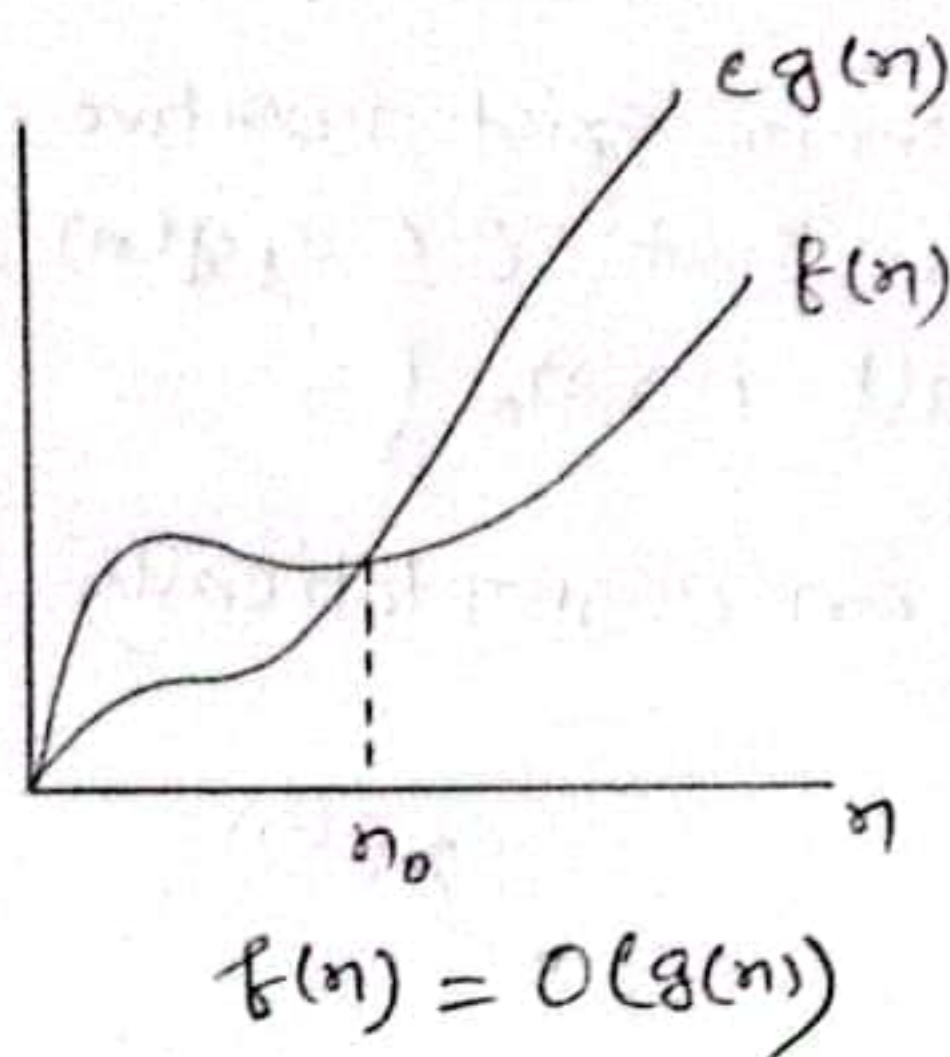
$$f(n) = \Theta(g(n))$$

O Notation

For a given function $g(n)$, we denote by $O(g(n))$ the set of functions

$$O(g(n)) = \{ f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n > n_0 \}$$

We say that $g(n)$ is an asymptotically upper bound for $f(n)$.

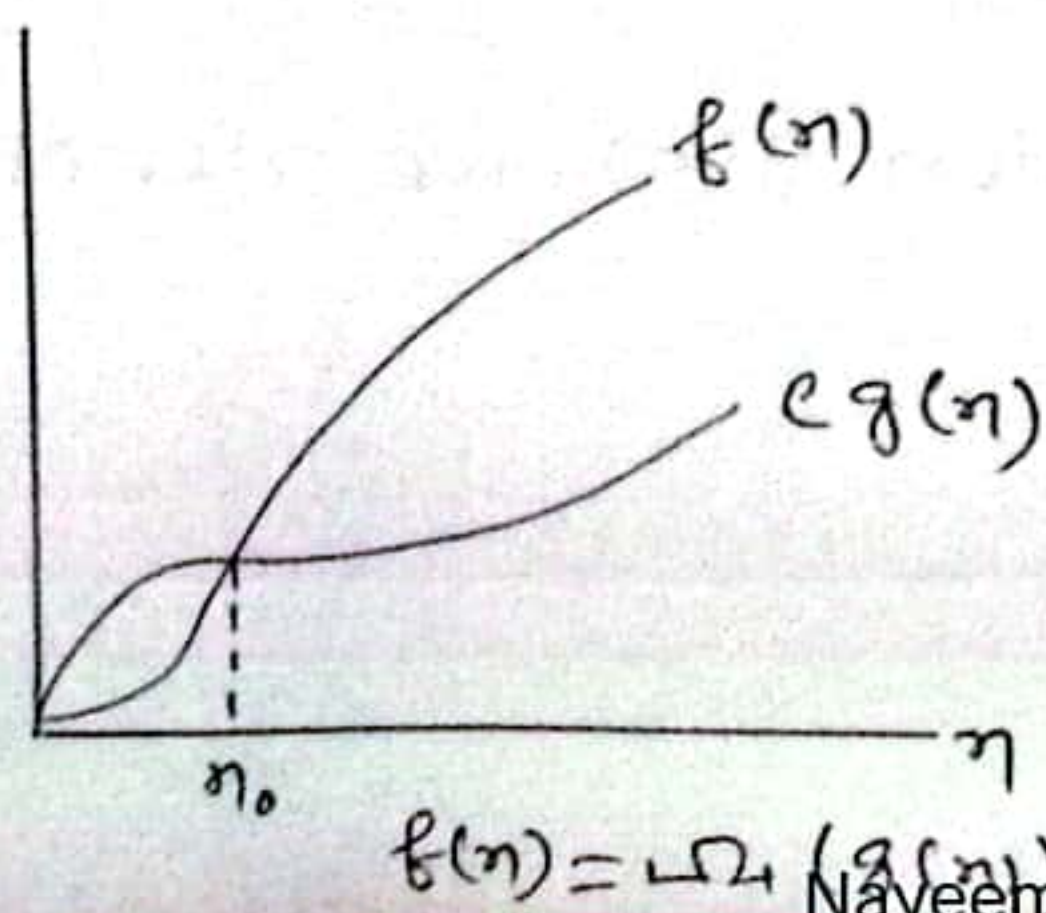


Ω Notation

For a given function $g(n)$, we denote by $\Omega(g(n))$ the set of functions

$$\Omega(g(n)) = \{ f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n > n_0 \}$$

We say that $g(n)$ is an asymptotically lower bound for $f(n)$



Q1 Show that $an^3 + bn + c \neq O(n^3)$

For the purpose of contradiction, let us assume that

$$an^3 + bn + c = O(n^3)$$

This implies that there exist positive numbers c_1, c_2 and n_0 such that

$$0 \leq c_1 n^3 \leq an^3 + bn + c \leq c_2 n^3 \text{ for all } n > n_0.$$

So, dividing by n^2 , we get

$$c_1 n \leq a + \frac{b}{n} + \frac{c}{n^2} \leq c_2 n$$

The relation on the right side i.e.

$$a + \frac{b}{n} + \frac{c}{n^2} \leq c_2 n \text{ holds for very large}$$

values of n . But the left sided relation

$$c_1 n \leq a + \frac{b}{n} + \frac{c}{n^2}$$

doesn't hold for large values of n . Hence, $an^3 + bn + c \neq O(n^3)$ is proved by contradiction.

Q2 Show that $4n^3 + 5n \neq O(n^2)$

For the purpose of contradiction, let us assume that

$$4n^3 + 5n = O(n^2)$$

This implies that there exist positive numbers c_1, c_2 and n_0 such that

$$0 \leq c_1 n^2 \leq 4n^3 + 5n \leq c_2 n^2 \text{ for all } n > n_0.$$

So, dividing by n^2 , we get

$$c_1 \leq 4n + \frac{5}{n} \leq c_2$$

The relation on left side i.e. $c_1 \leq 4n + \frac{5}{n}$ holds for large values of n .

But the relation on right side i.e. $4n + \frac{5}{n} \leq c_2$ doesn't hold for large values of n since c_2 is a constant.

So, our assumption is not true at all. Hence it is proved by contradiction that

$$4n^3 + 5n \neq O(n^2)$$

Dynamic Programming

Q Write down the sequences of 4 steps to develop a dynamic programming.

1. Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution.
3. Compute the value of an optimal solution, typically in a bottom-up fashion.
4. Construct an optimal solution from computed information.

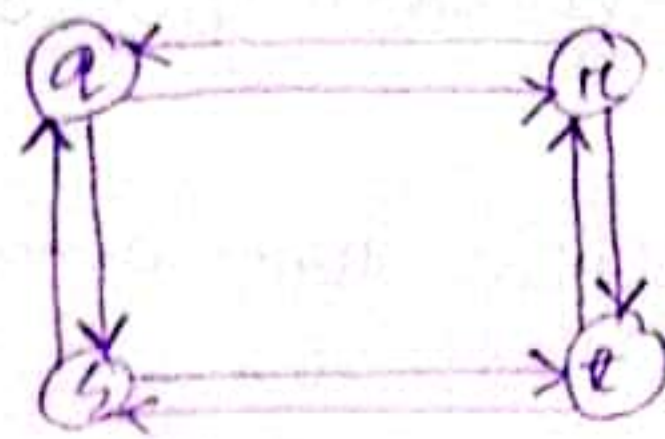
Q What do you mean by optimization problems? Which of the following are optimization problems? Give justification in support of your answer.

- i) Choosing a path to go from Dhaka to Seoul so that no sea is crossed.
- ii) Finding a way to go from Chittagong to Rangpur with minimum cost.
- iii) Finding a way to best utilize the class rooms of the department.
- iv) Choosing the maximum from a list of numbers.
- v) Finding an empty class room from the list of class rooms.

Q Optimization Problem: Those Problems are called optimization problems which can have many possible solution having a value and someone wish to find a solution with the optimal (minimum or maximum) value.

- i) Not an optimization problem since the problem doesn't exhibit optimal substructure property.
- ii) It's an optimization problem since it can be divided into sub-problems.
- iii) It's an optimization problem. There exists more than one possible solution.
- iv) Not an optimization problem. Exact solution is possible.
- v) It's an optimization problem. More than one solution is possible.

III What is optimal substructure? Taking the following graph as an example, show that 'longest simple path' problem doesn't have optimal substructure property.



Optimal Sub-structure: If an optimal solution to a problem contains within it optimal solutions to its sub-problems, then it is said that the given problem exhibits optimal substructure property.

The path $q \rightarrow r \rightarrow t$ is a longest simple path from q to t . But neither the sub-path $q \rightarrow r$ is not a longest simple path from q to r , nor is the sub-path $r \rightarrow t$ a longest simple path from r to t .

Since the supposed optimal solution doesn't contain itself the solutions to its sub-problems, the given 'longest simple path' doesn't have optimal sub-structure property.

Q Write down two similarities and two dissimilarities between divide and conquer and dynamic programming paradigms.

Similarities:

1. Both Paradigms divide a given problem into some sub-problem and solve the sub-problems.
2. Both Paradigms combine the solutions to the sub-problems to obtain the final solution.

Dissimilarities:

1. In dynamic programming, sub-problems may be overlapped and those overlapped sub-problems are solved once and then stored in a table for future look up. But in divide and conquer paradigm, sub-problems may or may not be ~~overlapped~~ overlapped. If does, it solves them everytime i.e. it doesn't use memoization.
2. In divide and conquer, the sub-problems are independent of each other while in case of dynamic programming, the sub-problems are not independent i.e. solution to one sub-problem may be required to solve another sub-problem.

Q1 What do you mean by overlapping sub-problems?

When a recursive algorithm revisits the same problem repeatedly, it is said that the optimization problem has overlapping sub-problems.

Q2 Why Divide and Conquer is applied instead of DP when a problem doesn't have overlapping sub-problem property?

In dynamic programming, a sub-problem is solved as a part of one bigger sub-problem which might be ~~req~~ required to be solved again as a part of another subproblem. So, the very first time, a sub-problem is solved and its result are tabulated/stored and the next time it is encountered, instead of solving it again, the result is looked up in the table using constant time per lookup.

Since, memoization isn't needed when a problem doesn't have overlapping sub-problem property, divide and conquer is used to solve it instead of DP.

Q) Consider the following function :

$$F(n) = F(\lfloor n/2 \rfloor) \times F(\lfloor n/4 \rfloor) \quad ; n > 0$$

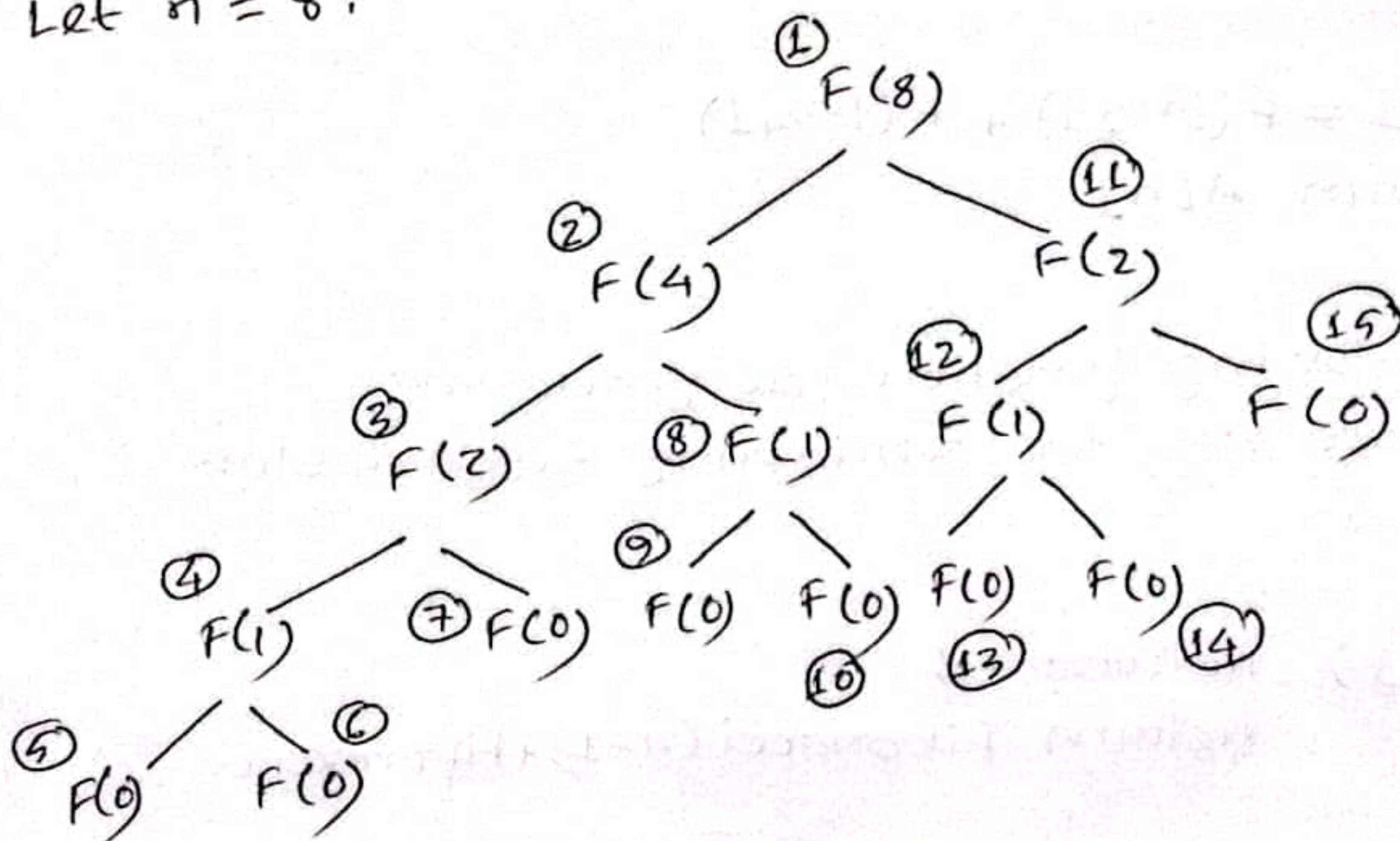
$$F(n) = 1 \quad ; n = 0$$

i) Why should you use DP to evaluate this function?

ii) Write a dynamic programming solution/algorithm to evaluate this function.

i) To better understand the usefulness of DP to solve the function, let us assume a value for n and examine the recursive calling tree.

Let $n = 8$.



We can see that there is total 15 function calls for $n = 8$. If we examine, it is noticeable that number 4, 8 and 12 are the same calling. That applies for number 3 and 11. These callings are redundant. What if we could save the result for $F(2)$ that was got from number 3? In that case, we don't have to calculate $F(2)$ again while it is called again in number 11 calling. Hence, we should use DP to evaluate

this function,

ii)

Let $A[1..n]$ be an array which can be accessed globally and is set 0 to all the elements initially.

$F(n)$:

1. if $n == 0$
2. return 1
3. else if $A[n] != 0$
4. return $A[n]$
5. else
6. $A[n] = F(\lfloor n/2 \rfloor) + F(\lfloor n/4 \rfloor)$
7. return $A[n]$

Q Show that the following function for generating Fibonacci sequence has the overlapping sub-problem property.

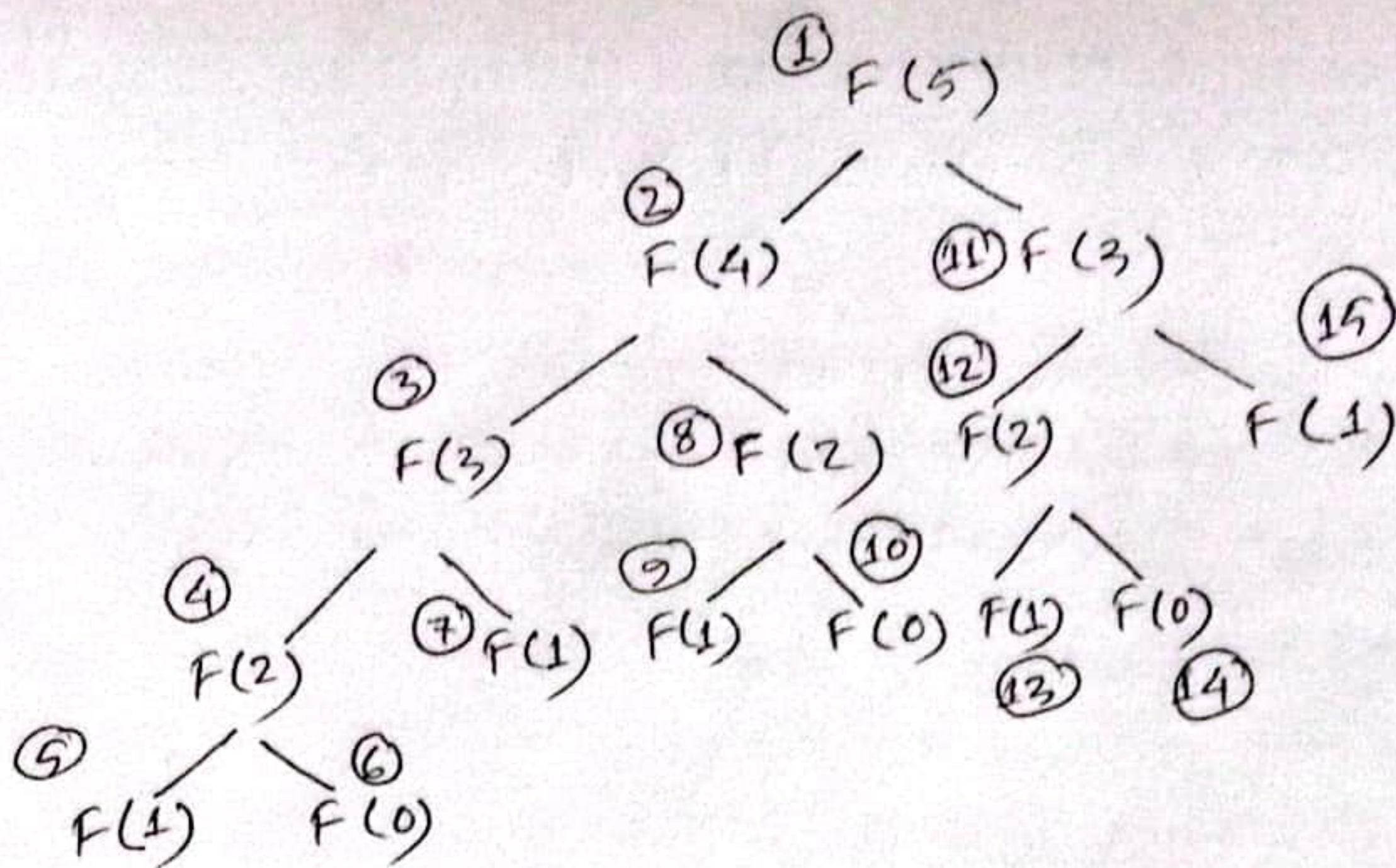
Fibonacci(n)

if ($n \leq 1$) return n ;

else return Fibonacci($n-1$) + Fibonacci($n-2$);

How can you solve the problem using DP?

For $n = 5$, the recursive callings tree be like below;



We can notice that $F(2)$ is called thrice (function call number: 4, 8, 12). Hence according to definition and the recursive tree above, the given problem has overlapping sub-problem property.

DP Solⁿ:

Let $A[1..n]$ be a global array and is set 0 to all the elements initially.

Fibonacci(n):

1. if $n \leq 1$
2. return n
3. else if $A[n] \neq 0$
4. return $A[n]$
5. else
6. $A[n] = \text{Fibonacci}(n-1) + \text{Fibonacci}(n-2)$
7. return $A[n]$

Matrix chain Multiplication

▣ The required function to calculate the minimum cost

$$m[i, j] = \begin{cases} 0 & ; i=j \\ \min_{i \leq k < j} \{ m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \} & ; i < j \end{cases}$$

▣ Given a sequence $P = \{3, 2, 4, 5, 6\}$ which represents the chain of matrices such that i -th matrix A_i is of dimension $p_{i-1} \times p_i$. Fill up the following m and s tables for computing the cost of matrix chain multiplication using dynamic programming for the matrix chain A .

matrix dimension		A_1	A_2	A_3	A_4
		3×2	2×4	4×5	5×6
		$p_0 \ p_1$	$p_1 \ p_2$	$p_2 \ p_3$	$p_3 \ p_4$

m

		i	1	2	3	4
j	4		136	100	120	0
	3		70	40	0	
	2		24	0		
	1		0			

s

		i	1	2	3	4
j	4		1	3	3	
	3		1	2		
	2		1			
	1					

$m[1, 2]$ →

When $k=1$,

$$m[1, 2] = m[1, 1] + m[2, 2] + p_0 p_1 p_2$$

$$= 0 + 0 + (3 \times 2 \times 4)$$

$$= 24$$

and $s[1, 2] = 1$

$$\underline{m[2,3]}$$

When $k=2$,

$$\begin{aligned} m[2,3] &= m[2,2] + m[3,3] + P_1 P_2 P_3 \\ &= 0 + 0 + (2 \times 4 \times 5) \\ &= 40 \end{aligned}$$

$$\text{and } s[2,3] = 2 \quad [\because k=2]$$

$$\underline{m[3,4]}$$

When $k=3$,

$$\begin{aligned} m[3,4] &= m[3,3] + m[4,4] + P_2 P_3 P_4 \\ &= 0 + 0 + (4 \times 5 \times 6) \\ &= 120 \end{aligned}$$

$$\text{and } s[3,4] = 3$$

$$\underline{m[1,3]}$$

When $k=1$,

$$\begin{aligned} m[1,3] &= m[1,1] + m[2,3] + P_0 P_1 P_3 \\ &= 0 + 40 + (3 \times 2 \times 5) \\ &= 70 \end{aligned}$$

When $k=2$,

$$\begin{aligned} m[1,3] &= m[1,2] + m[3,3] + P_0 P_2 P_3 \\ &= 24 + 0 + (3 \times 4 \times 5) \\ &= 84 \end{aligned}$$

$$\therefore m[1,3] = 70 \text{ when } k=1$$

$$\text{and } s[1,3] = 1$$

$m[2,4]$

When $k=2$,

$$\begin{aligned}m[2,4] &= m[2,2] + m[3,4] + p_1 p_2 p_4 \\&= 0 + 120 + (2 \times 4 \times 6) \\&= 168\end{aligned}$$

When $k=3$,

$$\begin{aligned}m[2,4] &= m[2,3] + m[4,4] + p_1 p_3 p_4 \\&= 40 + 0 + (2 \times 5 \times 6) \\&= 100\end{aligned}$$

$$\therefore m[2,4] = 100$$

$$\text{and } s[2,4] = 3$$

$m[1,4]$

When $k=1$,

$$\begin{aligned}m[1,4] &= m[1,1] + m[2,4] + p_0 p_1 p_4 \\&= 0 + 100 + (3 \times 2 \times 6) \\&= 136\end{aligned}$$

When $k=2$,

$$\begin{aligned}m[1,4] &= m[1,2] + m[3,4] + p_0 p_2 p_4 \\&= 24 + 120 + (3 \times 4 \times 6) \\&= 216\end{aligned}$$

When $k=3$,

$$\begin{aligned}m[1,4] &= m[1,3] + m[4,4] + p_0 p_3 p_4 \\&= 70 + 0 + (3 \times 5 \times 6)\end{aligned}$$

$$= 160$$

$$\therefore m[1,4] = 136$$

$$\text{and } s[1,4] = 1$$

Hence, the minimal cost is ~~160~~ $m[1,4] = 136$.

Optimal Parenthesization for the given matrices:

$$(A_1)((A_2 \times A_3)(A_4))$$

Q1 Recursively express the cost of multiplying a chain of matrices.

The recursive expression is

$$m[i,j] = \begin{cases} 0 & ; i=j \\ \min_{i \leq k < j} \{ m[i,k] + m[k+1,j] + \\ P_{i-1}P_kP_j & ; i < j \end{cases}$$

Q2 Given a sequence $P = \{2, 2, 3, 3, 5\}$ which represents the chain of matrices such that i -th matrix A_i is of dimension $P_{i-1} \times P_i$

i) Fill up m and s tables.

ii) why the table s is needed?

i)

matrix	A_1	A_2	A_3	A_4
dimension	2×2	2×3	3×3	3×5
	$P_0 P_1$	$P_1 P_2$	$P_2 P_3$	$P_3 P_4$

m	i	1	2	3	4
j	4	60	48	45	0
	3	30	18	0	
	2	12	0		
	1	0			

m	i	1	2	3
j	4	3	3	3
	3	1	2	
	2	1		

$m[1,3]$

When $k=1$,

$$\begin{aligned}
 m[1,3] &= m[1,1] + m[2,3] + P_0 P_1 P_3 \\
 &= 0 + 18 + (2 \times 2 \times 3) \\
 &= 30
 \end{aligned}$$

When $k=2$,

$$\begin{aligned}
 m[1,3] &= m[1,2] + m[3,3] + P_0 P_2 P_3 \\
 &= 12 + 0 + (2 \times 3 \times 3) \\
 &= 30
 \end{aligned}$$

$$\therefore m[1,3] = 30 \text{ and } s[1,3] = 1$$

$m[2,4]$

When $k=2$,

$$\begin{aligned}
 m[2,4] &= m[2,2] + m[3,4] + P_1 P_2 P_4 \\
 &= 0 + 45 + (2 \times 3 \times 5) \\
 &= 75
 \end{aligned}$$

When $k=3$,

$$\begin{aligned}m[2,4] &= m[2,3] + m[4,4] + p_1 p_3 p_4 \\&= 18 + 0 + (2 \times 3 \times 5) \\&= 48\end{aligned}$$

$$\therefore m[2,4] = 48 \text{ and } s[2,4] = 3$$

$m[1,4]$,

When $k=1$,

$$\begin{aligned}m[1,4] &= m[1,1] + m[2,4] + p_0 p_1 p_4 \\&= 0 + 48 + (2 \times 2 \times 5) \\&= 68\end{aligned}$$

When $k=2$,

$$\begin{aligned}m[1,4] &= m[1,2] + m[3,4] + p_0 p_2 p_4 \\&= 12 + 45 + (2 \times 3 \times 5) \\&= 87\end{aligned}$$

When $k=3$,

$$\begin{aligned}m[1,4] &= m[1,3] + m[4,4] + p_0 p_3 p_4 \\&= 30 + 0 + (2 \times 3 \times 5) \\&= 60\end{aligned}$$

$$\therefore m[1,4] = 60 \text{ and } s[1,4] = 3$$

Hence, the optimal cost is $m[1,4] = 60$.

Optimal Parenthesization

$$((A_1) (A_2 \times A_3))(A_4)$$

ii)

The s table records the value for k when $m[i, j]$ is minimum. From that table, we can easily determine where to put parentheses.

Ex: Given a sequence $P = \{30, 35, 15, 5, 10, 20, 25\}$ which represents the chain of matrices such that i -th matrix A_i is of dimension $P_{i-1} \times P_i$.

i) Calculate $m[3, 6]$ and $s[3, 6]$.

ii) Find an optimal parenthesization of matrix chain product for above matrices.

matrix	A_1	A_2	A_3	A_4	A_5	A_6
dimension	30×35	35×15	15×5	5×10	10×20	20×25
	$P_0 P_1$	$P_1 P_2$	$P_2 P_3$	$P_3 P_4$	$P_4 P_5$	$P_5 P_6$

		i					
		1	2	3	4	5	6
j	6	15,125	10,500	5,375	3,500	5,000	0
	5	11,875	7,125	2,500	1,000	0	
	4	9,375	4,375	750	0		
	3	7,875	2,625	0			
	2	15,750	0				
	1	0					

	1	2	3	4	5
6	3	3	3	5	5
5	3	3	3	4	
4	3	3	3		
3	1	2			
2	1				

i)

$m[3,6]$,

when $k=3$,

$$\begin{aligned}
 m[3,6] &= m[3,3] + m[4,6] + P_2 P_3 P_6 \\
 &= 0 + 3500 + (15 \times 5 \times 25) \\
 &= 5375
 \end{aligned}$$

when $k=4$,

$$\begin{aligned}
 m[3,6] &= m[3,4] + m[5,6] + P_2 P_4 P_6 \\
 &= 750 + 5000 + (15 \times 10 \times 25) \\
 &= 9500
 \end{aligned}$$

when $k=5$,

$$\begin{aligned}
 m[3,6] &= m[3,5] + m[6,6] + P_2 P_5 P_6 \\
 &= 2500 + 0 + (15 \times 20 \times 25) \\
 &= 10000
 \end{aligned}$$

$\therefore m[3,6] = 5375$ and

$s[3,6] = 3$

ii)

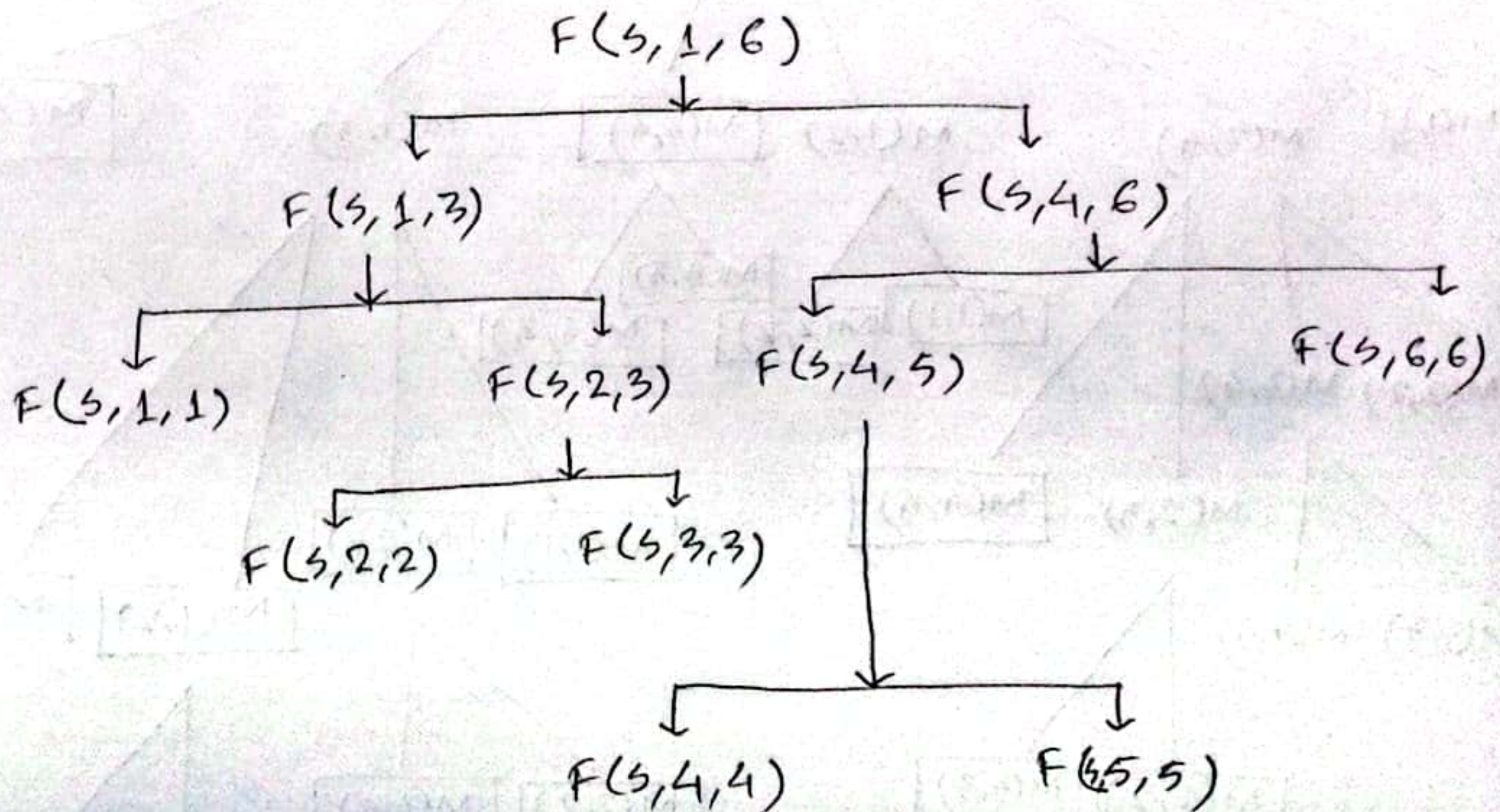
$$\left((A_1 \quad (A_2 \times A_3)) \left((A_4 \times A_5) \quad A_6 \right) \right)$$

④ The recursive procedure to find parenthesization

$F(s, i, j)$:

1. if $i == j$
2. Print " A_i "
3. else
4. Print "("
5. $F(s, i, s[i, j])$
6. $F(s, s[i, j] + 1, j)$
7. Print ")"

The recursive calling tree be

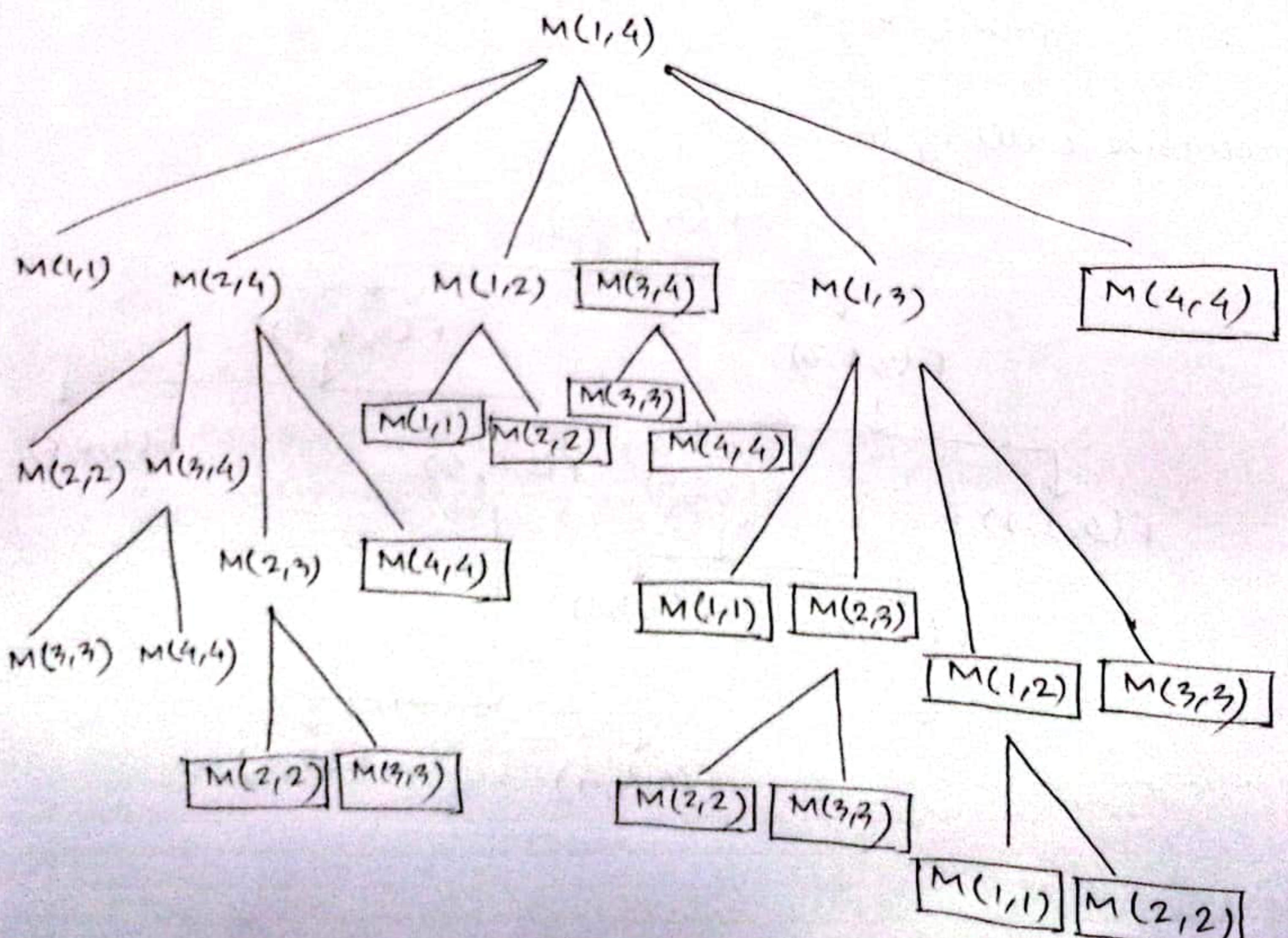


▣ Suppose, you need to find the optimal parenthesization of a matrix chain product. Find a recursive solution and write the appropriate equation(s).

[see above]

▣ Show that the matrix chain multiplication problem has overlapping sub-problem property.

Let us consider $M(i, n)$ be a recursive procedure which calculates the minimum number of scalar multiplications needed for a matrix chain $\langle A_1, A_2, \dots, A_n \rangle$. The recursive calling tree for $M(1, 4)$ be



The redundant callings are boxed. We can see that $M(1,2)$ was called for finding solutions for both $M(1,4)$ and $M(1,3)$.

Hence, we can say that this problem has overlapping sub-problem property.

Longest Common Subsequence

Q Write down the recursive formula for the optimal sub-structure of the LCS problem.

$$c[i, j] = \begin{cases} 0 & ; \text{if } i=0 \text{ or } j=0 \\ c[i-1, j-1] + 1 & ; \text{if } i, j > 0 \text{ and } x_i = y_j \\ \max(c[i, j-1], c[i-1, j]) & ; \text{if } j > 0 \text{ and } x_i \neq y_j \end{cases}$$

Q Find the LCS of two strings $X = \text{"ADEFGIED"}$ and $Y = \text{"ACBFHED"}$ by generating LCS table. Using that table, tell the LCS length of ("ADEFGIE" , "ACBF") and ("ADEFGIE" , "ACBFHE")

Length Table

		0	1	2	3	4	5	6	7
	Y_j		A	C	B	F	H	E	D
0	X_i		0	0	0	0	0	0	0
1	A	0	1	← 1	← 1	← 1	← 1	← 1	← 1
2	D	0	↑ 1	↑ 1	↑ 1	↑ 1	↑ 1	↑ 1	↖ 2
3	C	0	↑ 1	2	← 2	← 2	← 2	← 2	↑ 2
4	F	0	↑ 1	↑ 2	↑ 2	3	← 3	← 3	← 3
5	G	0	↑ 1	↑ 2	↑ 2	↑ 3	↑ 3	↑ 3	↑ 3
6	E	0	↑ 1	↑ 2	↑ 2	↑ 3	↑ 3	4	← 4
7	F	0	↑ 1	↑ 2	↑ 2	↑ 3	↑ 3	↑ 4	↑ 4
8	D	0	↑ 1	↑ 2	↑ 2	↑ 3	↑ 3	↑ 4	5

The LCS of the two given strings is "ACFED" of length 5.

LCS Length of "ADEFGHE" and "ACBF"

The cell $C[6,4]$ gives us the length which is 3.
[The LCS is "ACF"]

LCS length of "ADCFHE" and "ACBFHE"

The cell $c[6,6]$ from above table gives us the length which is 4.

[The LCS is "ACFE"]

Find LCS of $X = \{a, b, a, d, c, b\}$ and $Y = \{b, c, a, d, e\}$ using DP.

LCS Table

		0	1	2	3	4	5
	Y_j		(b)	c	(a)	(d)	(e)
0	X_i	0	0	0	0	0	0
1	a	(0)	↑ 0	↑ 0	↖ 1	← 1	← 1
2	(b)	0	↖ 1	↖ 1	↑ 1	↑ 1	↑ 1
3	(a)	0	↑ 1	↑ 1	↖ 2	← 2	← 2
4	(d)	0	↑ 1	↑ 1	↑ 2	↖ 3	← 3
5	(e)	0	↑ 1	↖ 2	↑ 2	↑ 3	↖ 4
6	b	0	↖ 1	↑ 2	↑ 2	↑ 3	↑ 4

The LCS of the given strings is " $\{b, a, d, e\}$ " of length 4.

Ex X = "ACOFFE", Y = "ADCEFFE".

- i) Generate LCS table.
- ii) Using the table, mention what is the LCS of X & Y?
- iii) Using the table, can you find the LCS of "ACO" & "ADCEFFE"?

LCS Table

		0	1	2	3	4	5	6
	Y_j		(A)	D	(C)	F	(F)	(E)
0	X_i	(0)	0	0	0	0	0	0
1	(A)	0	(↖ ₁)	(↖ ₁)	← ₁	← ₁	← ₁	← ₁
2	(C)	0	↑ ₁	↑ ₁	(↖ ₂)	(↖ ₂)	← ₂	← ₂
3	0	0	↑ ₁	↑ ₁	↑ ₂	(↑ ₂)	↑ ₂	↑ ₂
4	(F)	0	↑ ₁	↑ ₁	↑ ₂	↖ ₃	(↖ ₃)	← ₃
5	(E)	0	↑ ₁	↑ ₁	↑ ₂	↑ ₃	↑ ₃	(↖ ₄)

ii) LCS of X and Y is "ACFE" of length 4.

iii) The LCS of "ACD" and "ADCEFE" is "AC" is of length 2 which can be found from the cell $C[3, 6]$.

Q Do you think that the LCS problem has overlapping sub-problem property? Justify your answer using suitable example.

Let us consider two strings be $X = \{x_1, x_2, \dots, x_m\}$ and $Y = \{y_1, y_2, \dots, y_n\}$. To find LCS of X and Y , we should examine either one or two sub-problems.

If $x_m = y_n$, we must find an LCS of X_{m-1} and Y_{n-1} . Appending $x_m = y_n$ to this LCS yields LCS of X and Y .

If $x_m \neq y_n$, then we must solve two subproblems:

1. finding an LCS of X_{m-1} and Y
2. finding an LCS of X and Y_{n-1}

Whichever of these two LCSs is longer is an LCS of X and Y . Because these cases exhaust all possibilities.

We can readily see the overlapping sub-problem property in the LCS problem. To find an LCS of X and Y , we may need to find the LCS of X and Y_{n-1} and of X_{m-1} and Y . But each of these subproblems has the subproblems of finding an LCS of X_{m-1} and Y_{n-1} . Many other subproblems share subproblems.

QUICK SORT

Algorithm

The key to this algorithm is the partition procedure, which rearranges the sub-array $A[p..r]$ in place.

PARTITION (A, p, r)

1. $x = A[r]$
2. $i = p - 1$
3. for $j = p$ to $r - 1$
4. if $A[j] \leq x$
5. $i = i + 1$
6. exchange $A[i]$ with $A[j]$
7. exchange $A[i + 1]$ with $A[r]$
8. return $i + 1$

QUICKSORT (A, p, r)

1. if $p < r$
2. $q = \text{PARTITION}(A, p, r)$
3. $\text{QUICKSORT}(A, p, q - 1)$
4. $\text{QUICKSORT}(A, q + 1, r)$

Q Do you think the Quick Sort algorithm follows divide and conquer process? Defend your answer.

Yes. Quick Sort algorithm follows Divide-and-Conquer Process.

Defense: Here is the three steps.

Divide: It partitions the array $A[p..r]$ into two (possibly empty) sub-arrays $A[p..q-1]$ and $A[q+1..r]$ such that each element of $A[p..q-1]$ is less than or equal to $A[q]$, which in turn, less than or equal to each element of $A[q+1..r]$. It computes the index q as part of this partitioning procedure.

Conquer: It sorts two sub-arrays $A[p..q-1]$ and $A[q+1..r]$ by recursive calls to Quick Sort.

Combine: Because the sub-arrays are already sorted, no work is needed to combine them; the entire array $A[p..r]$ is now sorted.

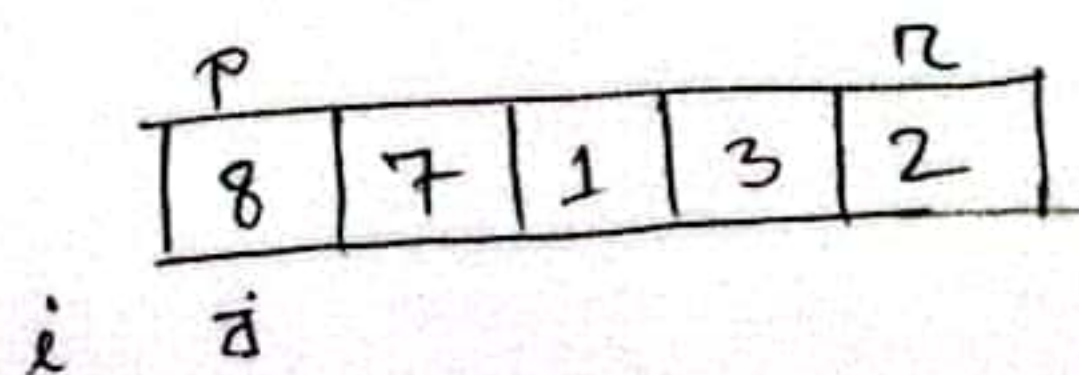
Q Show the operation of the PARTITION of the Quick Sort algorithm on the array

(i) $A = \langle 8, 7, 1, 3, 2 \rangle$

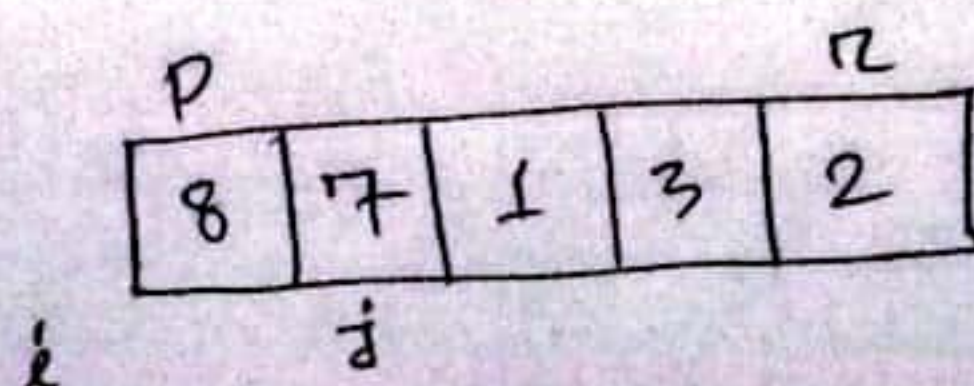
(ii) $A = \langle 8, 1, 7, 3, 2, 4, 6, 5 \rangle$

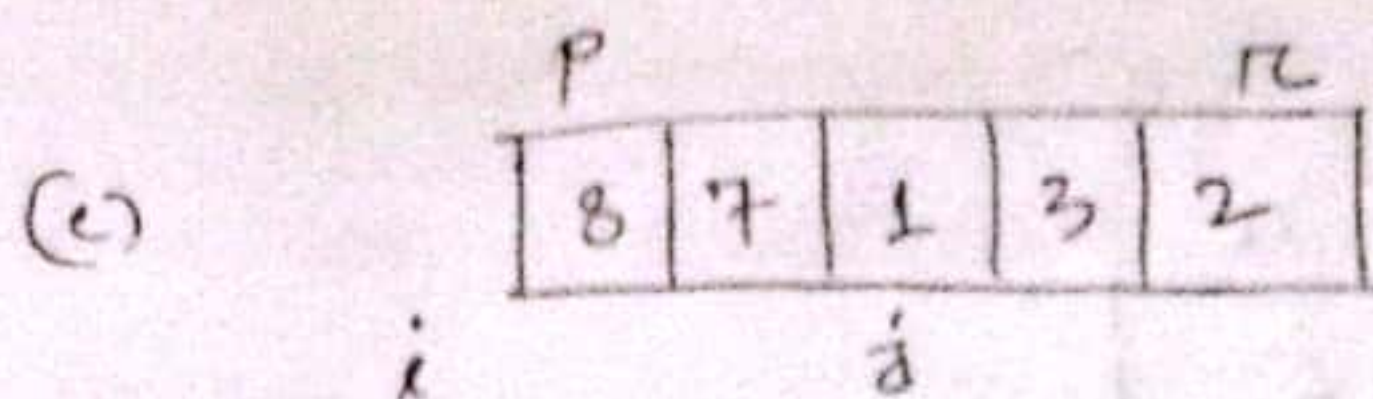
(i)

(a)

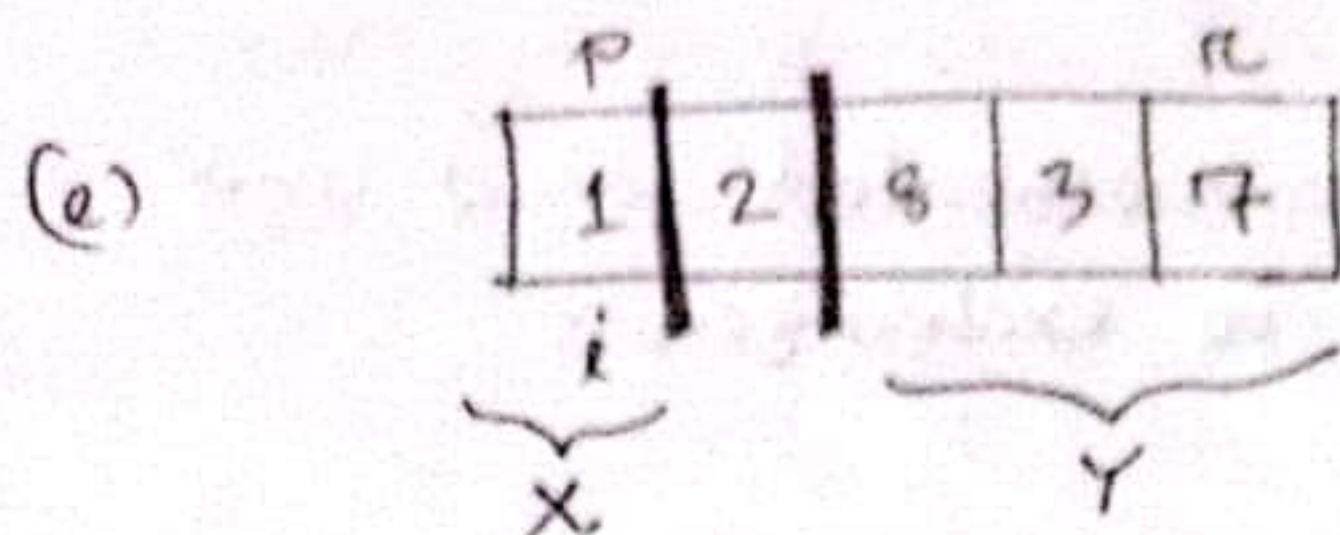
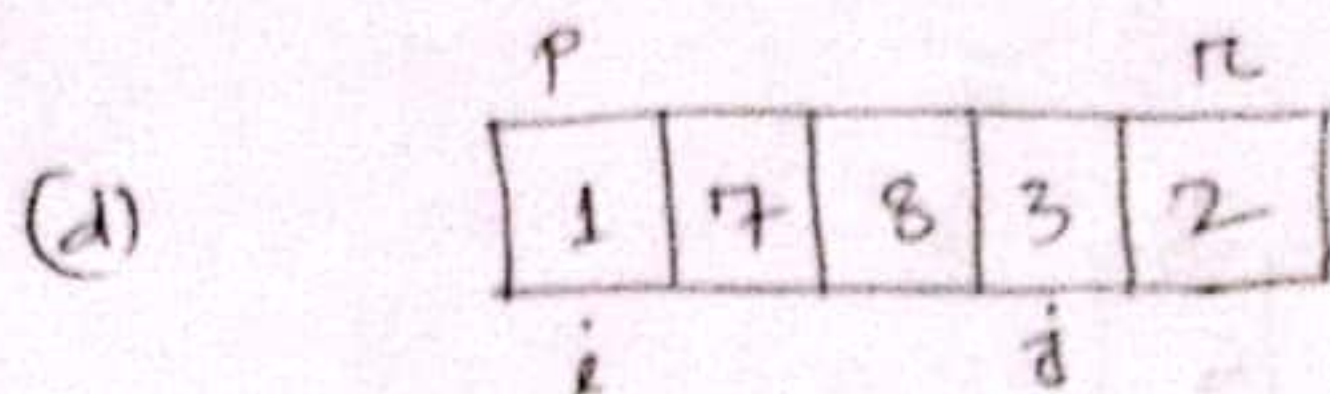


(b)



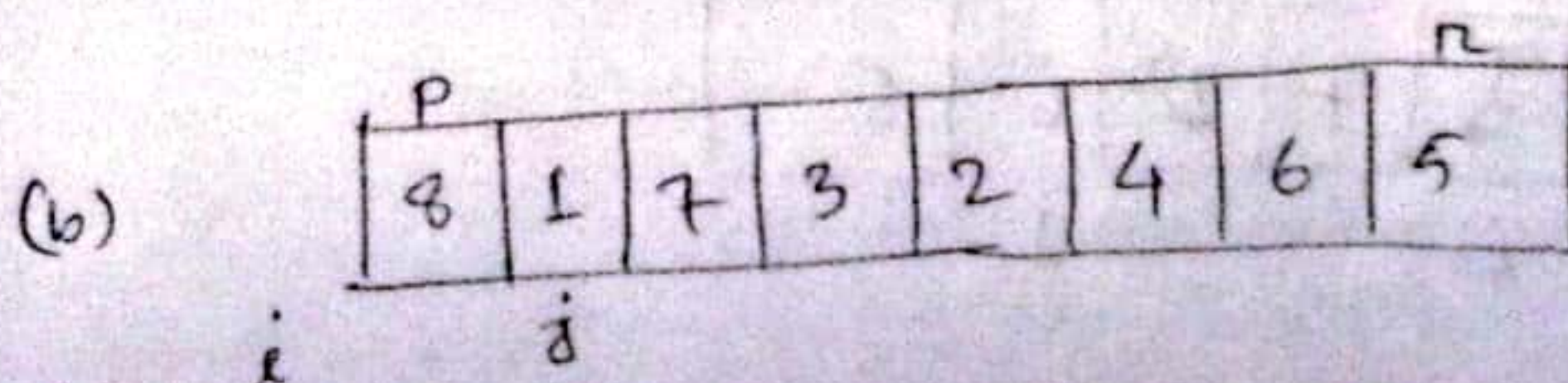
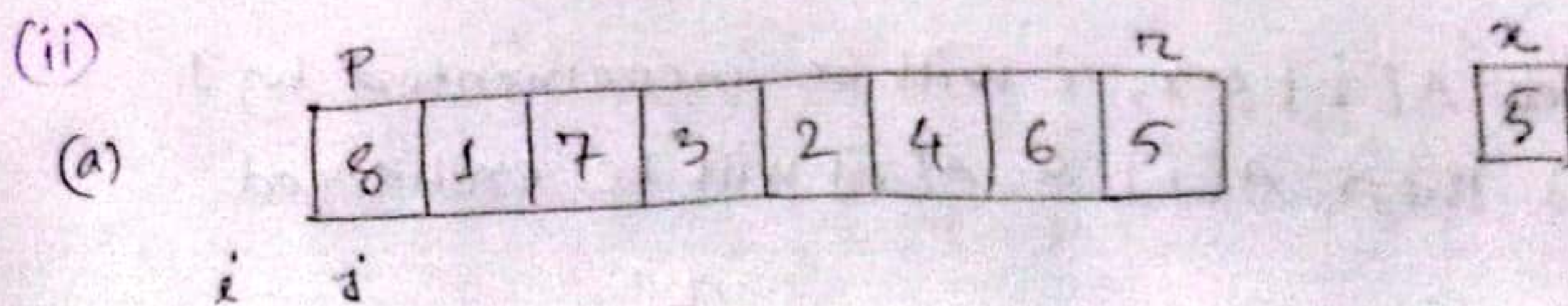


$\therefore A[j] \leq \pi$, then $A[i+1]$ and $A[j]$ will be exchanged

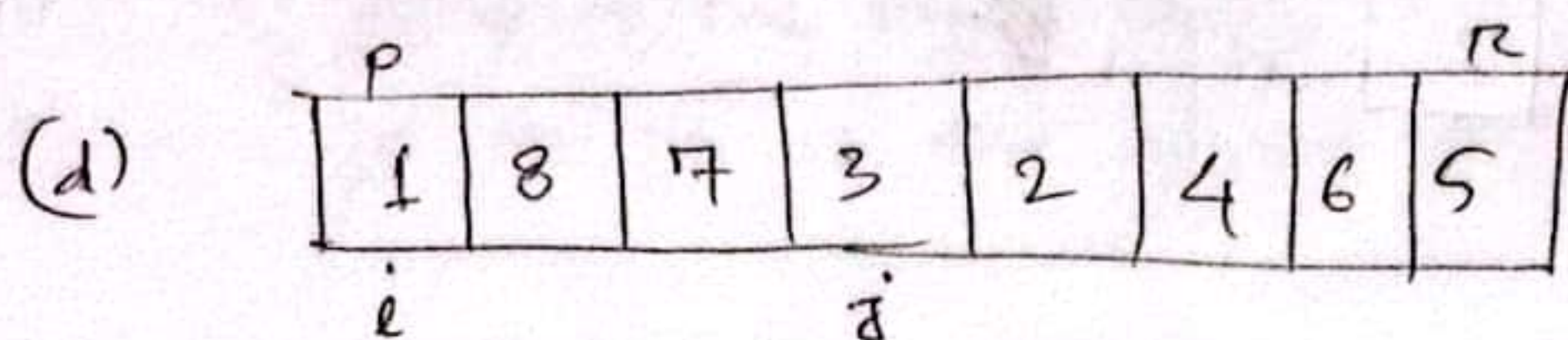
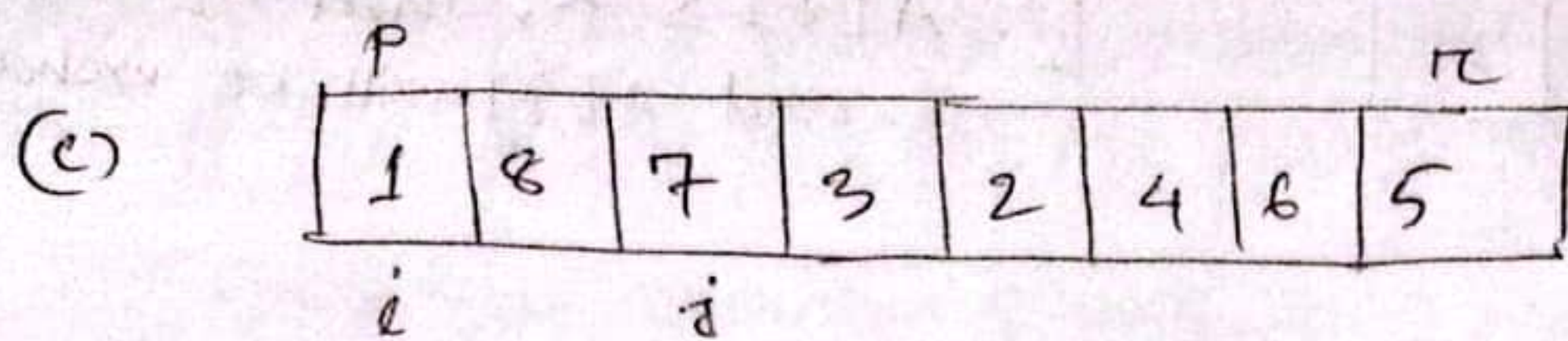


The partition operation on the given array is done. The pivot element $A[r] = 2$ is in its proper place as we can see that the elements on its left side are less than it and it is less than the elements on its right side.

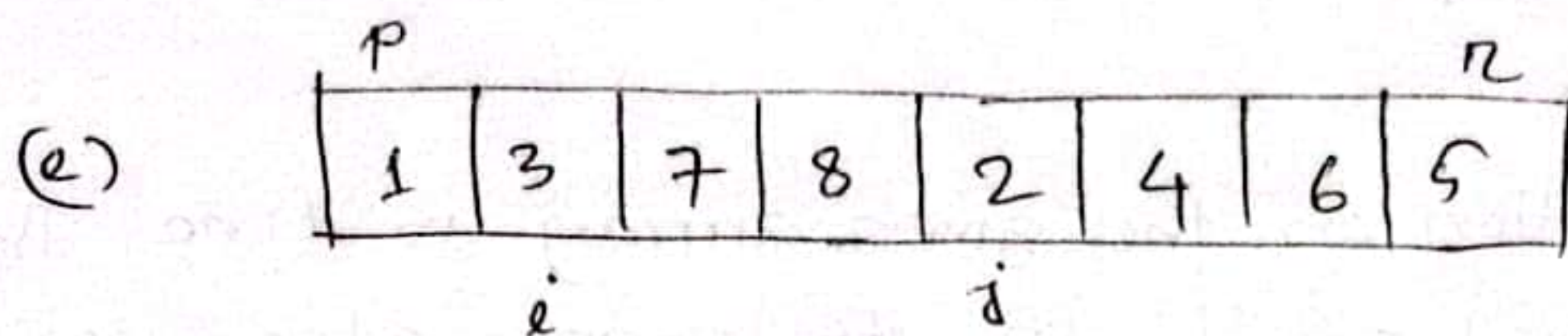
The call to PARTITION procedure will return 2 as the index of the pivot element is 2 here.



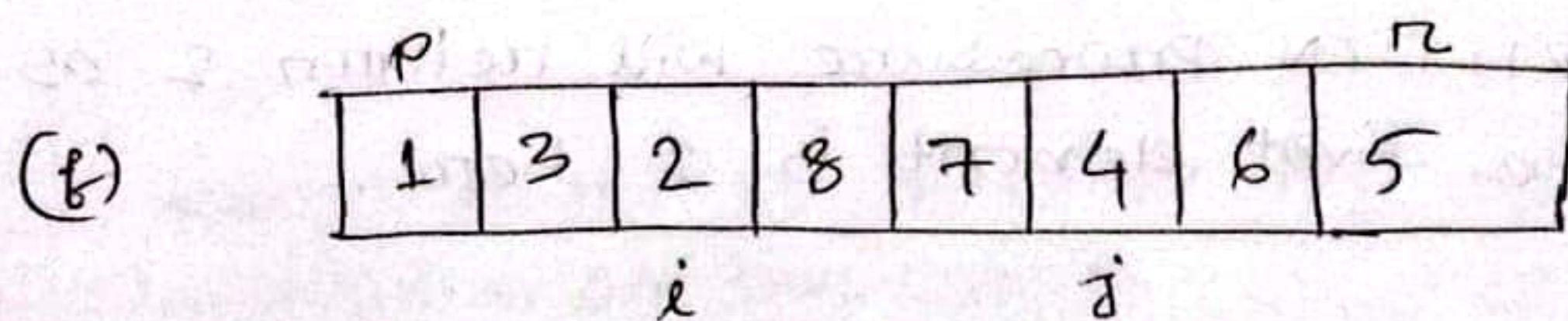
Since $A[j] \leq \pi$, $A[i+1]$ and $A[j]$ will be exchanged and i will be incremented by 1



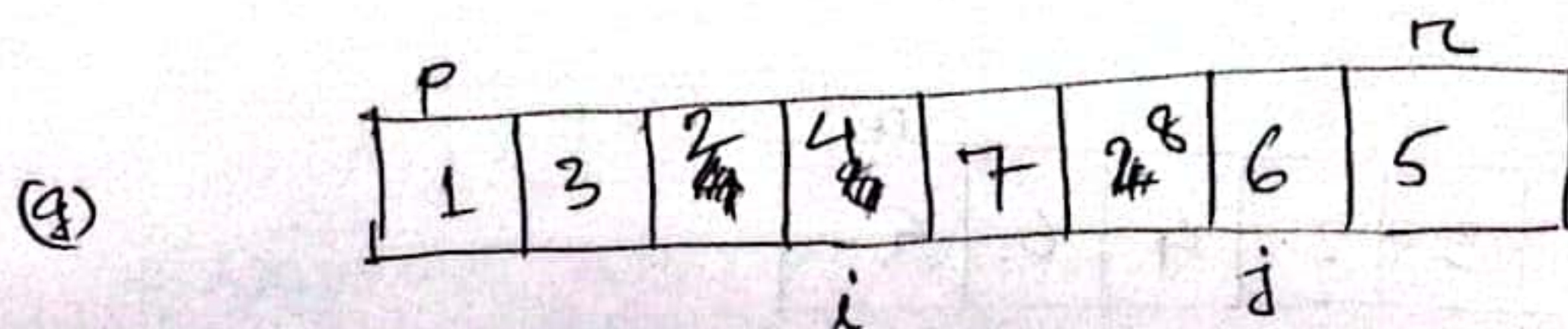
Since $A[j] \leq x$, i will be incremented by 1 and then $A[i]$ and $A[j]$ will be exchanged.



Since $A[j] \leq x$, i will be incremented by 1 and then $A[i]$ and $A[j]$ will be exchanged.

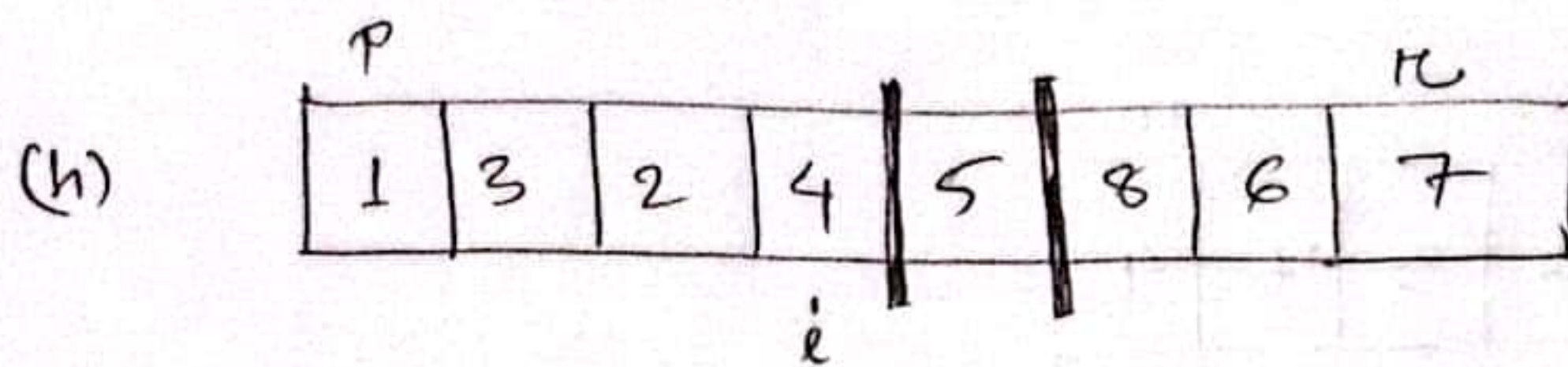


Since $A[j] \leq x$, i will be incremented by 1 and then $A[i]$ & $A[j]$ will be exchanged.



Now j will be incremented and the loop

will be terminated since $i = r$. Then $A[i+1]$ and $A[r]$ will be exchanged.



Since $A[j] \leq x$, i will be incremented and then $A[i]$ and $A[j]$ will be swapped.

(c)

p							r	
2	6	8	1	3	7	4		
i		j						

(d)

p							r	
2	6	8	1	3	7	4		
i		j						

Since $A[j] \leq x$, i will be incremented and then $A[i]$ and $A[j]$ will be swapped.

(e)

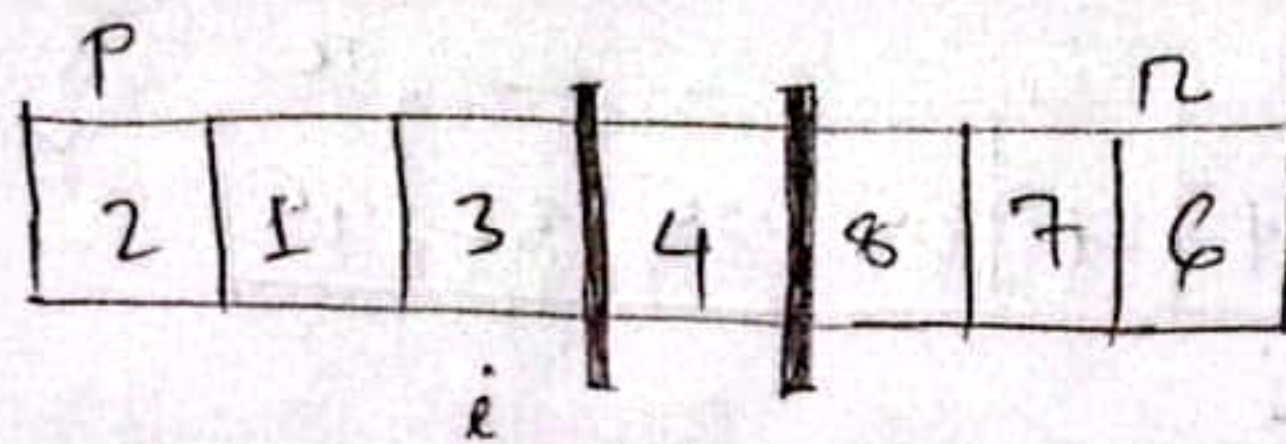
p							r	
2	1	8	6	3	7	4		
i		j						

Since $A[j] \leq x$, i will be incremented and then $A[i]$ & $A[j]$ will be swapped.

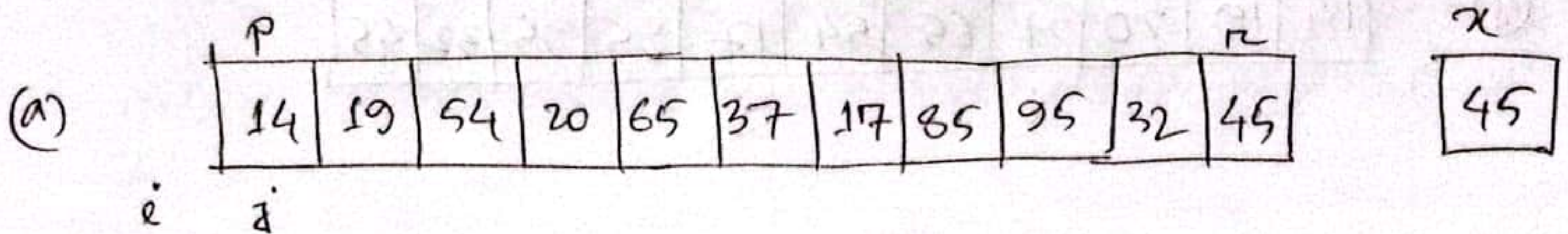
(f)

p							r	
2	1	3	6	8	7	4		
i		j						

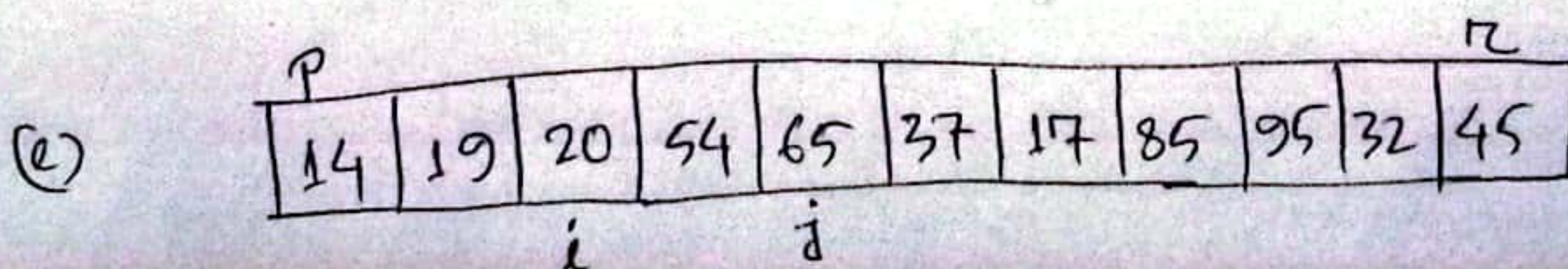
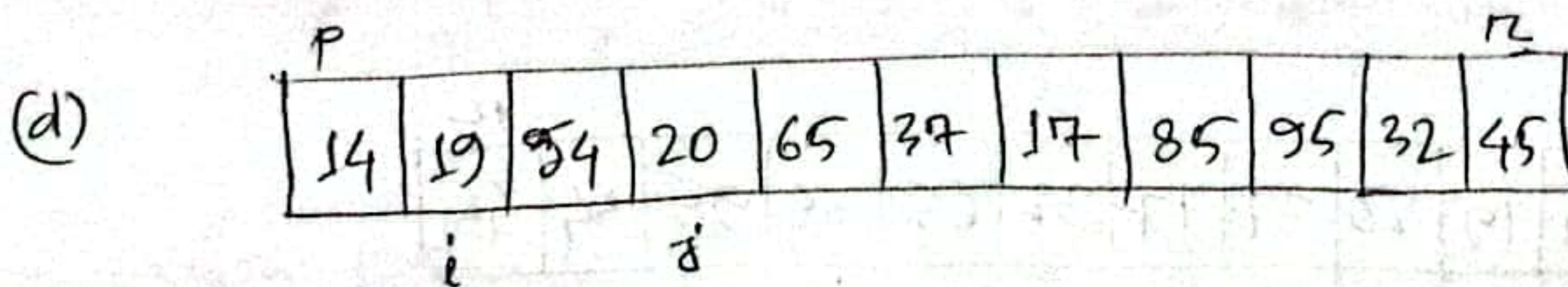
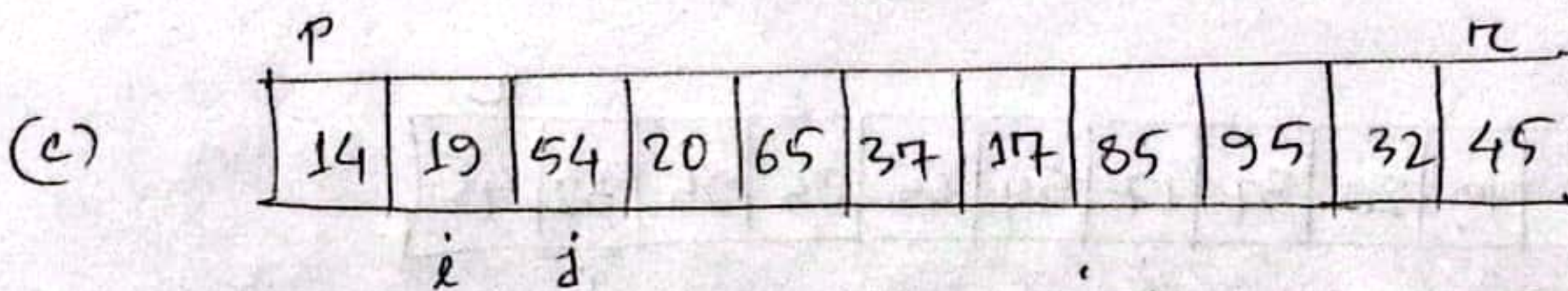
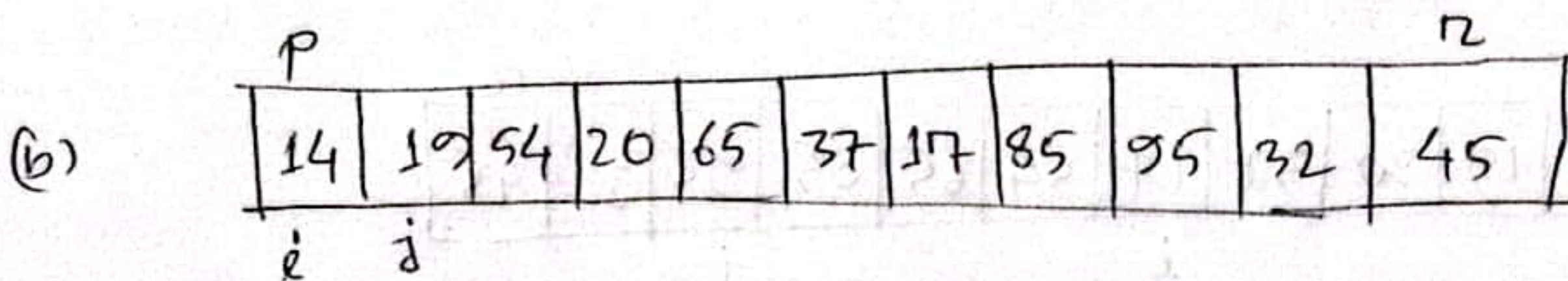
Now $A[i+1]$ and $A[r]$ will be exchanged and $(i+1)$ will be returned (i.e. 4).



(ii)



Since $A[i] \leq x$, i will be incremented and then $A[i]$ and $A[j]$ will be swapped. Here element 14 will be exchanged with itself.



(f)

P	14	19	20	54	65	37	17	85	95	32	45	R
			i			j						

(g)

P	14	19	20	37	65	54	17	85	95	32	45	R
				i			j					

(h)

P	14	19	20	37	17	54	65	85	95	32	45	R
					i			j				

(i)

P	14	19	20	37	17	54	65	85	95	32	45	R
					i			j				

(j)

P	14	19	20	37	17	54	65	85	95	32	45	R
					i					j		

(k)

P	14	19	20	37	17	32	65	85	95	54	45	R
						i						

p						r					
14	19	20	37	17	32	45	85	95	54	65	
					i						

Consider the following array $A[p..r] = \langle 5, 4, 1, 9, 5, 2, 7, 6, 7, 4, 3, 10 \rangle$. Array index begins at $p=5$ and ends at $r=16$. Assume pivot $x = A[r]$.

i) What value will be returned by partition procedure if run on array A?

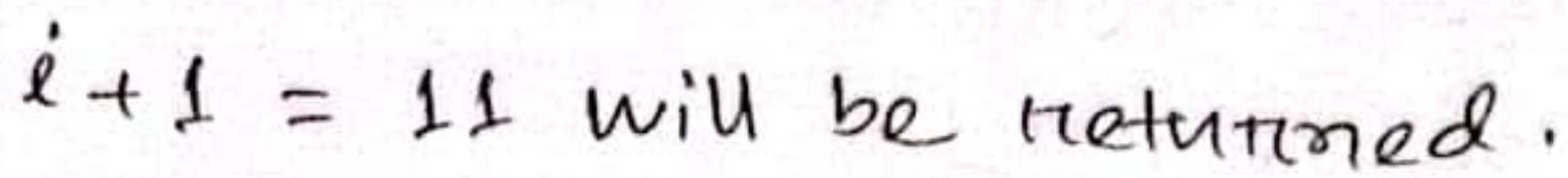
ii) What value will be returned if $A[p]$ is exchanged with $A[r]$ before calling Partition?

iii) Let $A[p..r] = \langle 2, 4, 1, x, x, 8, x, 6, 7, x, 3, 5 \rangle$ where x are some unknown values. What is the maximum and minimum value that could be returned by Partition?

(i)

	p											12
	5	4	1	9	5	2	7	6	7	4	3	10
i	i	i	i	i	i	i	i	i	i	i	i	
	i	i	i	i	i	i	i	i	i	i	i	

$i+1 = 16$ will be returned.



The maximum value could be returned if all the unknown values are less than or equal to 5 and the value could be 9.

1	2	3	4	5	6	7	8	9	10	11	12
2	4	1	3	X	X	X	X	5	8	6	7

1 2 3 4 5 6 7 8 9 10 11 12

2	4	1	3	5	X	X	X	X	8	6	7
---	---	---	---	---	---	---	---	---	---	---	---

Question: Suppose we are sorting an array of eight integers using Quick Sort algorithm and we have just finished the first partitioning with the array looking like this:

2 5 1 7 9 12 11 10

Which statement is correct?

1. The pivot could be either 7 or the 9.
2. The pivot could be the 7 but it's not the 9.
3. The pivot is not the 7, but it could be the 9.
4. Neither the 7 nor the 9 is the pivot.

Answer: Statement (1) is correct.

Explanation: All the numbers on the left side of 7 are smaller than 7 and all the numbers on the right side of 7 are greater than 7. This applies for 9 too.

So according to the definition of Pivot, either 7 or 9 could be the pivot.

Write down two advantages of Quick sort over Merge sort.

1. Auxiliary Space : Quick sort is an in-place sorting algorithm which takes no additional storage space to perform sorting while Merge Sort algorithm takes extra storage.
2. Locality of Reference: Quick sort in particular exhibits good cache locality and this makes it faster than Merge-Sort in many cases like in virtual memory environment.

Quicksort(A) // A is an array

1. if $P < r$
2. $q = \text{Partition}(A, P, r)$
3. Quicksort(A, P, $q-1$)
4. Quicksort(A, $q+1, r$)

What will happen if you delete line 1 (if $P < r$) from above algorithm? Explain

In simple terms, we can say stack overflow will happen.

Explanation: There will be such a situation when q will be 1. In that case, the third line might be executed as Quicksort(A, 1, 0) assuming $P = 1$. Since the condition at line 1 is removed, again q will have value 1 and again Quicksort(A, 1, 0)

will be called. This chain of recursive callings will continue until the stack is overflowed and eventually a Runtime Error ~~is~~ will ~~be~~ followed.

Q1 Write down the recurrence equation of the Quicksort algorithm in which partition ~~is~~ always produces a 9-to-1 proportional split.

$$T(n) = T\left(\frac{9n}{10}\right) + T\left(\frac{n}{10}\right) + cn$$

Q2 Why do we use randomized version of Quicksort? You may use an example to clarify your point.

Both the deterministic and randomized quicksort algorithms have the same best case running times of $O(n \log n)$ and the same worst case running times of $O(n^2)$. The difference is that with the deterministic algorithm, a particular input can elicit the worst case behavior. With the randomized version, however, no input can ~~also~~ always elicit the worst case behavior. The reason it matters is that, depending on how partitioning is implemented, an input that is already sorted or almost sorted, can elicit the worst case behavior in deterministic quicksort. That's why we use the randomized version of, Quicksort.

MERGE SORT

Algorithm

MERGE (A, P, q, r)

1. $n_1 = q - P + 1$
2. $n_2 = r - q$
3. Let $L[1..n_1+1]$ and $R[1..n_2+1]$ be new arrays
4. for $i = 1$ to n_1
5. $L[i] = A[P+i-1]$
6. for $j = 1$ to n_2
7. $R[j] = A[q+j]$
8. $L[n_1+1] = \infty$
9. $R[n_2+1] = \infty$
10. $i = 1$
11. $j = 1$
12. for $k = P$ to r
13. if $L[i] \leq R[j]$
14. $A[k] = L[i]$
 $i = i + 1$
15. else $A[k] = R[j]$
16. $j = j + 1$
- 17.

MERGE-SORT (A, P, R)

1. if $P < R$
2. $Q = \lfloor (P+R)/2 \rfloor$
3. MERGE-SORT (A, P, Q)
4. MERGE-SORT (A, Q+1, R)
5. MERGE (A, P, Q, R)

▣ Consider an array for merge sort :
 $A = \langle 5, 2, 3, 1, 6, 1, 7 \rangle$

- i) How many Merge-Sort call will be there when the initial call is on array A?
- ii) How many Merge operation will be there?

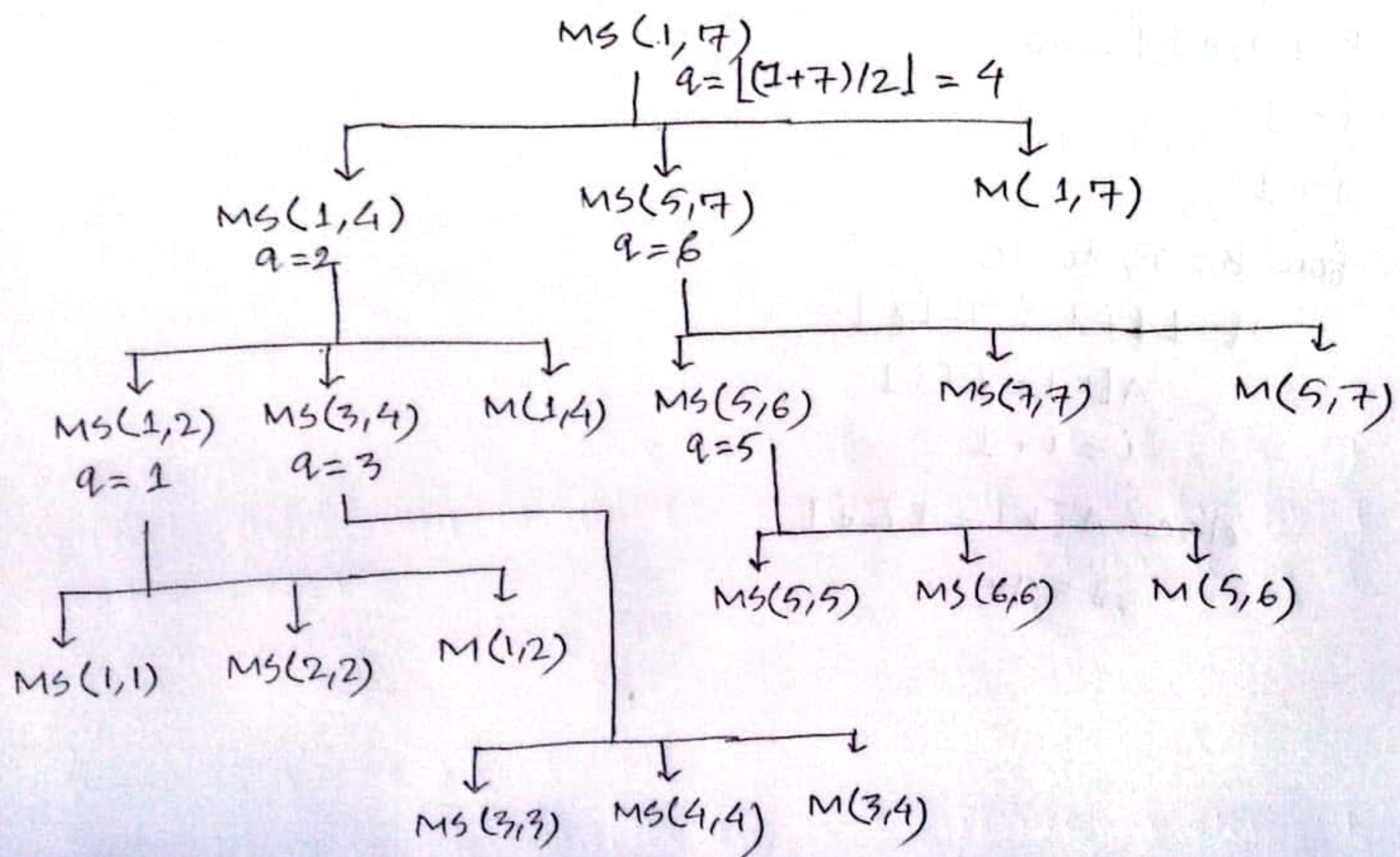
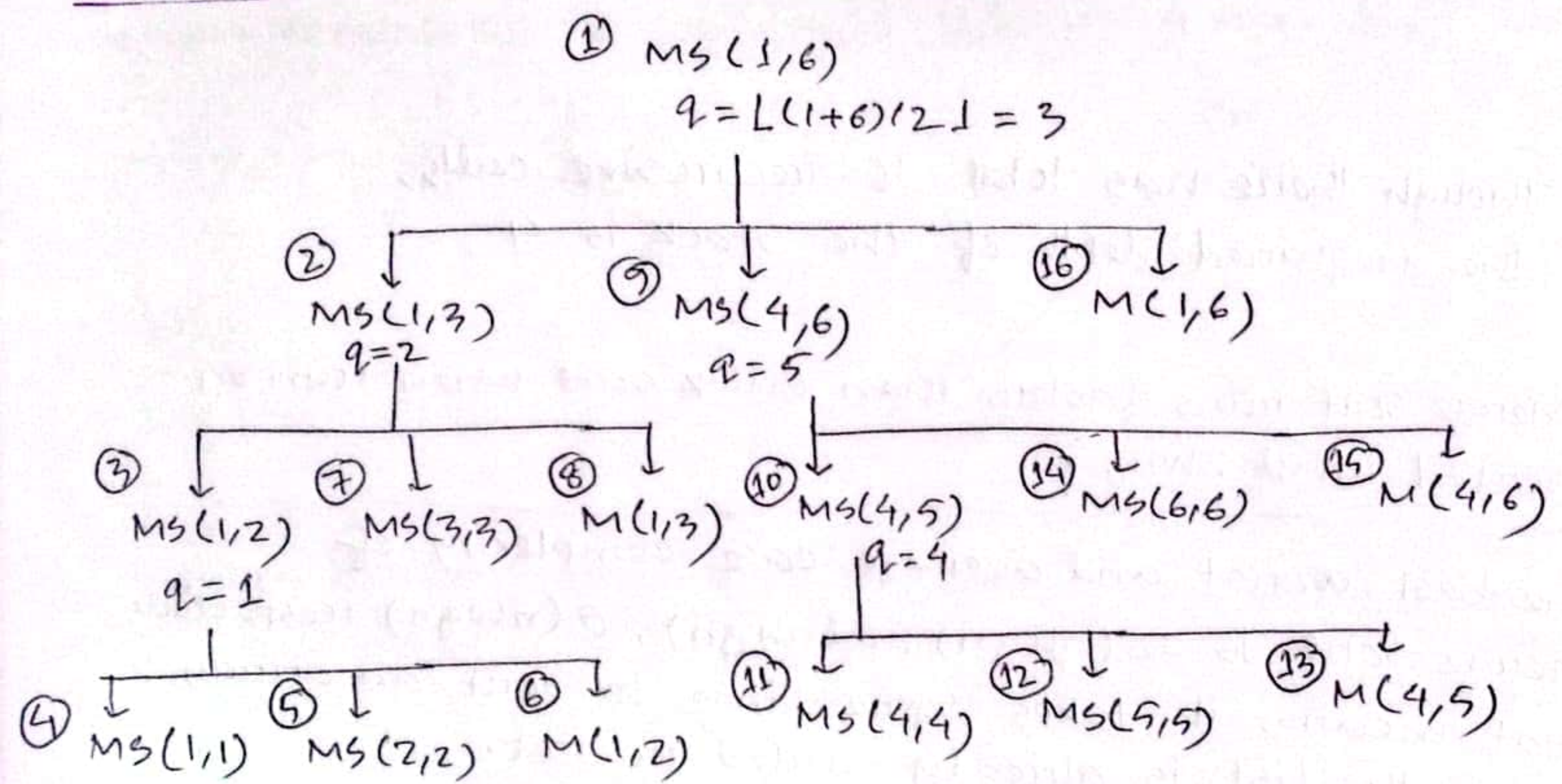


Fig: Recursive call tree for given array

- i) Total 13 calls including $MS(1,7)$
- ii) 6 Merge operation will be there.

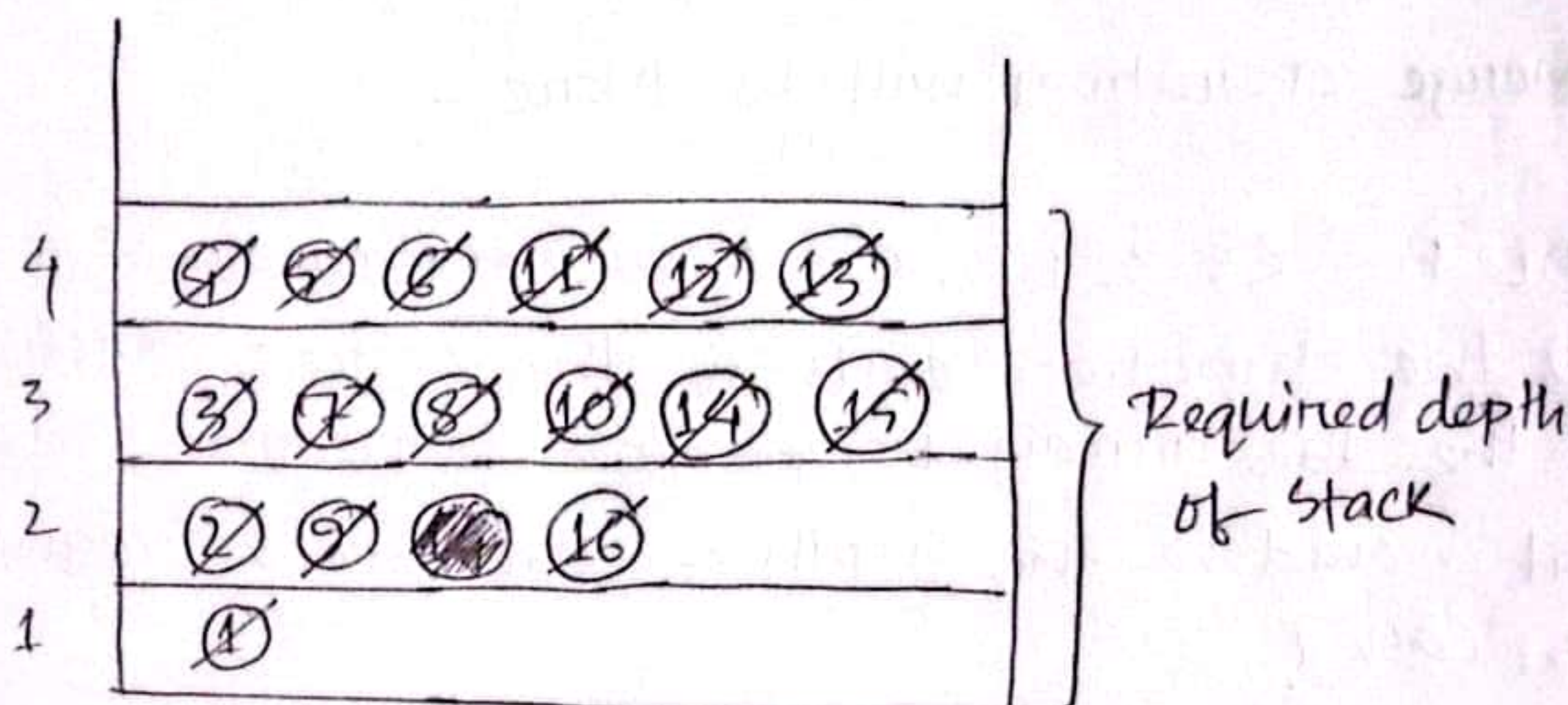
▣ Consider $A = \langle 8, 3, 1, 9, 5, 2 \rangle$ be an array for merge sort.

- i) Show the function calls of the merge sort algorithm for the ~~the~~ array using data structure stack.
- ii) What would be the depth of the stack required for this case?



The numberings inside circles are showing the order of recursive calls.

i)



ii) Though there was total 16 recursive calls, the required depth of the stack is 4.

☐ Merge sort runs faster than Quick sort when run on sorted input. Why?

The best, worst and average case complexity of Merge sort is $\Omega(n \log n)$, $O(n \log n)$, $\Theta(n \log n)$ respectively. That means, it takes same time to sort an array either the list is already sorted or not.

But in the case of Quick sort, if the list is already sorted, there will be $(n-1)$ comparisons for the first pivot, $(n-2)$ comparisons for the second pivot and so on. ~~In general, there will be total~~
We can express the recurrence as

$$T(n) = O(n) + T(n-1)$$

where $O(n)$ is the complexity to compare n elements. We can deduce that,

$$T(n) = O(n^2)$$

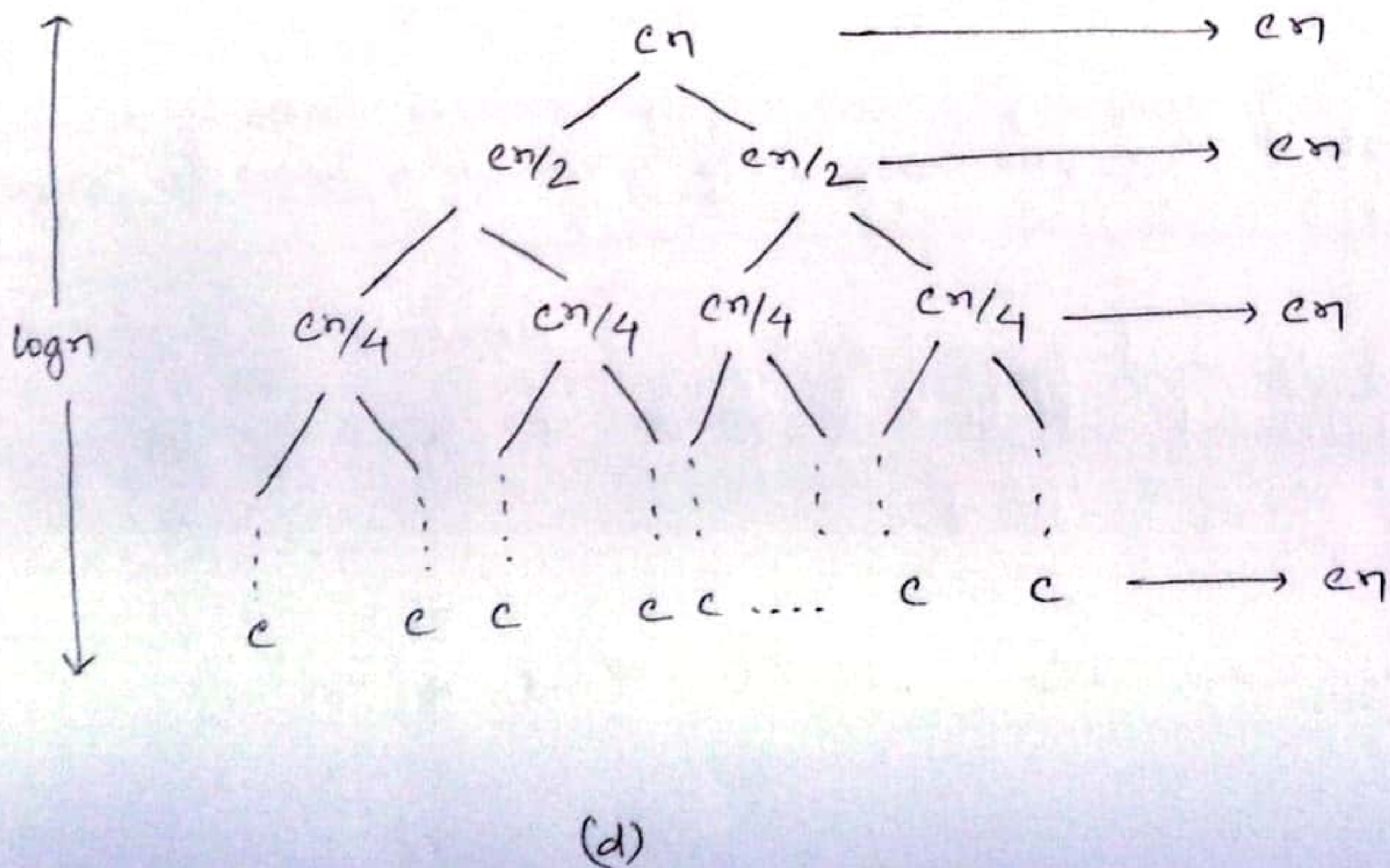
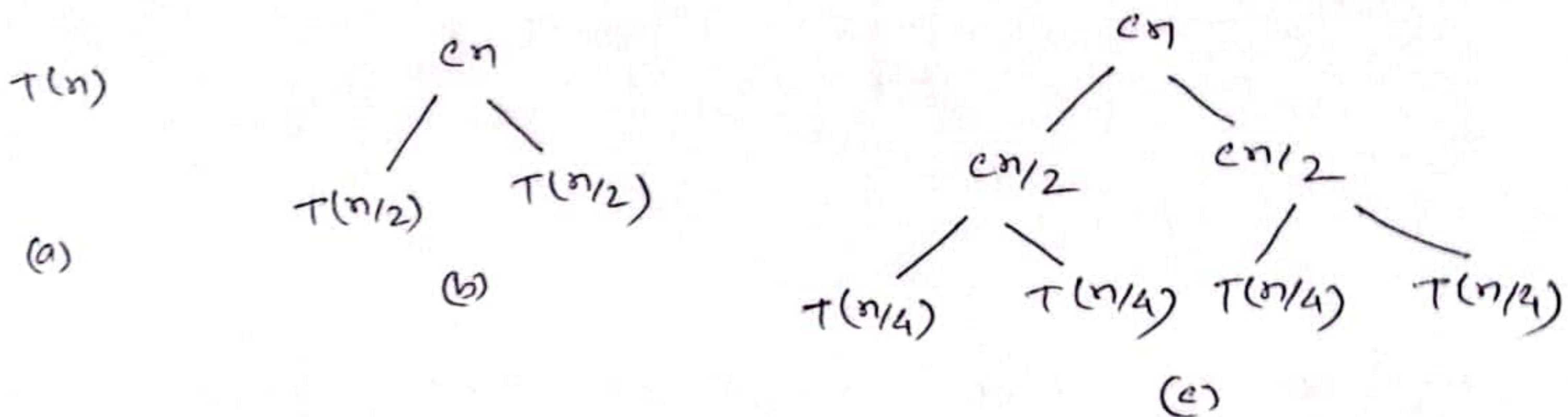
which is the worst case behavior of Quick sort algorithm.

▣ Construct a recursion tree for the worst case running time of the merge sort. Find out its total cost in term of θ notation.

The given recurrence equation is

$$T(n) = \begin{cases} c & \text{if } n=1 \\ 2T(n/2) + cn & \text{if } n > 1 \end{cases}$$

We can construct the recursion tree in 4 steps:



Step (a) shows $T(n)$ which progressively expands in steps (b) - (d) to form the recursion tree. The fully expanded tree in step (d) has $\log n + 1$ levels (i.e., it has height $\log n$, as indicated), and each level contributes a total cost of en .

The total cost, therefore is $en \log n + en$, which is $O(n \log n)$.

INSERTION SORT

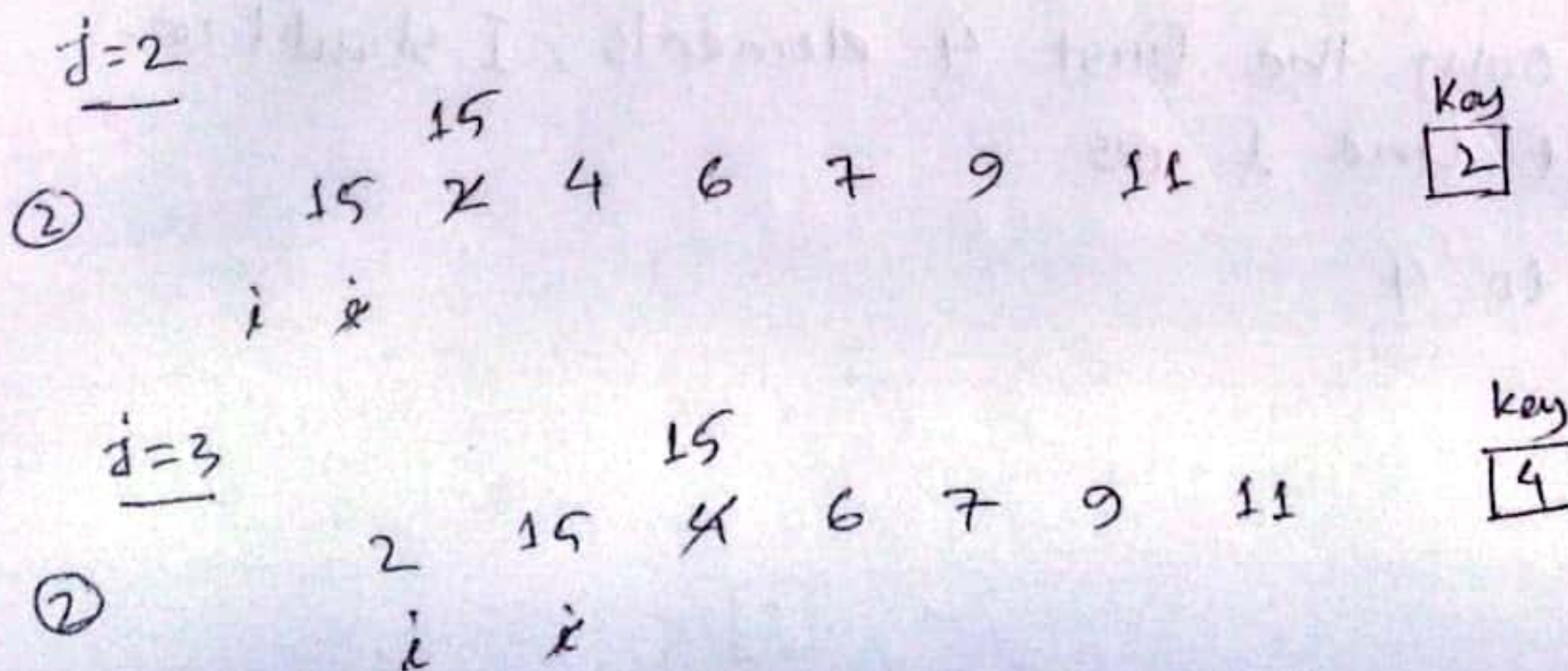
Algorithm

Insertion-Sort (A) // A is an array

1. for $j=2$ to A.length
2. key = A[j]
3. $i = j - 1$
4. while ($i > 0$) and ($A[i] > \text{key}$)
5. $A[i+1] = A[i]$
6. $i = i - 1$
7. $A[i+1] = \text{key}$

Given the algorithm of the Insertion Sort, how many times line 4 will be executed if run on the array $A = \langle 15, 2, 4, 6, 7, 9, 11 \rangle$?

Where should you change in the given algorithm if you want to sort only first four elements?



$$\underline{j=4}$$

② 2 4 15 ¹⁵
 _i _j ~~6~~ 7 9 11 key
6

$$\underline{j=5}$$

② 2 4 6 15 ¹⁵
 _i _j ~~7~~ 9 11 key
7

$$\underline{j=6}$$

② 2 4 6 7 15 ¹⁵
 _i _j ~~8~~ 11 key
9

$$\underline{j=7}$$

② 2 4 6 7 9 15 ¹⁵
 _i _j ~~11~~ key
11

So, the statement at line 4 will be executed for 12 times.

If I want to sort only the first 4 elements, I should re-write statement at line 1 as
 for $i=2$ to 4

Q4) Given the algorithm of Insertion sort,

i) why does the value of j at line 1 starts from 2?

ii) what would be the result if you rewrite line 4 as:
 $\text{while}(A[i] > \text{key})$?

iii) How many comparisons (i.e. $A[i] > \text{key}$ at line 4) will be there while you apply insertion sort on array $A = \langle 5, 1, 2, -3 \rangle$

iv) Calculate the running time of Insertion sort algorithm for the worst case.

i) A list (or sub-list) containing one element is always sorted. That's why, j starts from 2.

ii) If we omit the expression $(i > 0)$, lower bound of i will never be checked which will cause invalid array index accessing i.e. indexes less than 1 will be accessed; theoretically which don't exist but practically, key will be compared to some garbage values and the original list might be altered with those unexpected garbage values.

iii)

$j=2$

	5				
	5	1	2	-3	key [1]
i	i				

$[5] > [1] \checkmark$

$$\underline{j=3}$$

		5			key
1	5	2	-3		<u>2</u>
i	j				

$$\boxed{5} > \boxed{2} \checkmark$$

$$\boxed{1} > \boxed{2} \times$$

$$\underline{j=4}$$

		1	2	5	key
	1	2	5	-3	<u>-3</u>
i	j	k	l		

$$\boxed{5} > \boxed{-3} \checkmark$$

$$\boxed{2} > \boxed{-3} \checkmark$$

$$\boxed{1} > \boxed{-3} \checkmark$$

So, in total 6 comparison will take place.

iv)

Statements	Cost	Times
1	c_1	n
2	c_2	$n-1$
3	c_3	$n-1$
4	c_4	$\sum_{j=2}^n (t_j)$
5	c_5	$\sum_{j=2}^n (t_j - 1)$
6	c_6	$\sum_{j=2}^n (t_j - 1)$
7	c_7	$n-1$

Total running time,

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4 \sum_{j=2}^n t_j + c_5 \sum_{j=2}^n (t_j - 1) + c_6 \sum_{j=2}^n (t_j - 1) + c_7(n-1)$$

$$= c_1 n + c_2(n-1) + c_3(n-1) + c_4 \left(\frac{n(n+1)}{2} - 1 \right) +$$

$$c_5 \left(\frac{n(n-1)}{2} \right) + c_6 \left(\frac{n(n-1)}{2} \right) + c_7(n-1)$$

$$= c_1 n + c_2(n-1) + c_3(n-1) + c_4 \left(\frac{n^2}{2} + \frac{n}{2} - 1 \right)$$

$$+ c_5 \left(\frac{n^2}{2} - \frac{n}{2} \right) + c_6 \left(\frac{n^2}{2} - \frac{n}{2} \right) + c_7(n-1)$$

$$= \cancel{c_4} + \cancel{c_5} + \cancel{c_6} + \cancel{c_7} n^2$$

$$\therefore T(n) = \left(\frac{c_4}{2} + \frac{c_5}{2} + \frac{c_6}{2} \right) n^2 + (c_1 + c_2 + c_3 + \frac{c_4}{2} - \frac{c_5}{2} - \frac{c_6}{2} + c_7) n - (c_2 + c_4 + c_5 + c_7)$$

We can express this worst case running time as $an^2 + bn + c$ for constants a, b, c that again depend on the statement costs c_i ; it is thus a quadratic function of n .

Ex Let an array consists of the following elements:

1, 5, 10, 12, 15, 29

i) Is the arrangement defined as the "best case" for insertion sort?

ii) Is it the best case for quick sort?

i) As the list is already sorted, it can be defined as the best-case for insertion sort.

ii) As the list is already sorted, it can't be the best but the worst case for quick sort.

Q Assume that an array A contains 10000 numbers in sorted order. You are asked to sort them using Insertion-sort, Merge-sort and Quick-sort. Which algorithm will take minimum time and which will take maximum time? Justify your answer?

Insertion-sort algorithm will take minimum time. Because for each $j = 2, 3, \dots, n$, we can find that $A[i] \leq \text{key}$ where i has its initial value at $j-1$. So, there will be just one comparison operation for 1 number and we obtain the best case complexity as $\Omega(n)$.

Quick-sort algorithm will take maximum time. Since list is sorted, every pivot is in its proper place. However, comparisons and exchanging operations will take place for all pivots and we obtain the worst case complexity as $O(n^2)$.

In case of Merge-sort, any type of input takes the same amount of time and the average case complexity that we get is $O(n \log n)$.

Q When is Insertion-sort a good choice for sorting an array?

- i) Each component of the array requires a large amount of memory.
 - ii) Each component of the array requires a small amount of memory.
 - iii) The array has only a few items out of place.
 - iv) The processor speed is fast.
-

True statement is (iii). Insertion sort works better in sorted or almost sorted list.

Selection Sort

The Selection sort algorithm is given below:

Selection-Sort(A) // A is an array

1. for $j = 1$ to $A.length - 1$
2. $iMin = j$
3. for $i = j + 1$ to $A.length$
4. if $A[i] < A[iMin]$
5. $iMin = i$
6. if $(iMin \neq j)$
7. exchange $A[j]$ with $A[iMin]$

- i) Find how many times each line will execute for the input $A = \langle 15, 2, 4, 6, 7, 9, 11 \rangle$?
- ii) Express the time it takes on the input A in Θ -notation.
- iii) Intuitively tell what will be the worst case and best case running time.

i)	Statements	Times
	1	7
	2	6
	3	$\sum_{k=2}^7 K$
	4	$\sum_{k=2}^7 K - 1$
	5	$\sum_{k=2}^7 K - 1$

Statement	Times
6	6
7	6

This is not an exact calculation but approximate.

ii)

Statement	Times	Cost
1	n	c_1
2	$n-1$	c_2
3	$\sum_{k=2}^n k$	c_3
4	$\sum_{k=2}^n k - 1$	c_4
5	$\sum_{k=2}^n k - 1$	c_5
6	$n-1$	c_6
7	$n-1$	c_7

Total cost,

$$T(n) = c_1 n + c_2 (n-1) + c_3 \sum_{k=2}^n k + c_4 \left(\sum_{k=2}^n k - 1 \right) + c_5 \left(\sum_{k=2}^n k - 1 \right) + c_6 (n-1) + c_7 (n-1)$$

$$= c_1 n + c_2 (n-1) + c_3 \left(\frac{n(n+1)}{2} - 1 \right) + c_4 \left(\frac{n(n-1)}{2} \right) + c_5 \left(\frac{n(n-1)}{2} \right) + c_6 (n-1) + c_7 (n-1)$$

Nayeen Mahmood / 4AM / C161026

$$= c_1 n + c_2 (n-1) + c_3 \left(\frac{n^2}{2} + \frac{n}{2} - 1 \right) + c_4 \left(\frac{n^2}{2} - \frac{n}{2} \right) \\ + c_5 \left(\frac{n^2}{2} - \frac{n}{2} \right) + c_6 (n-1) + c_7 (n-1)$$

$$= \left(\frac{c_3}{2} + \frac{c_4}{2} + \frac{c_5}{2} \right) n^2 + \left(c_1 + c_2 + \frac{c_3}{2} + \frac{c_4}{2} - \frac{c_5}{2} + c_6 + c_7 \right) n \\ - (c_2 + c_3 + c_6 + c_7)$$

We can write this as

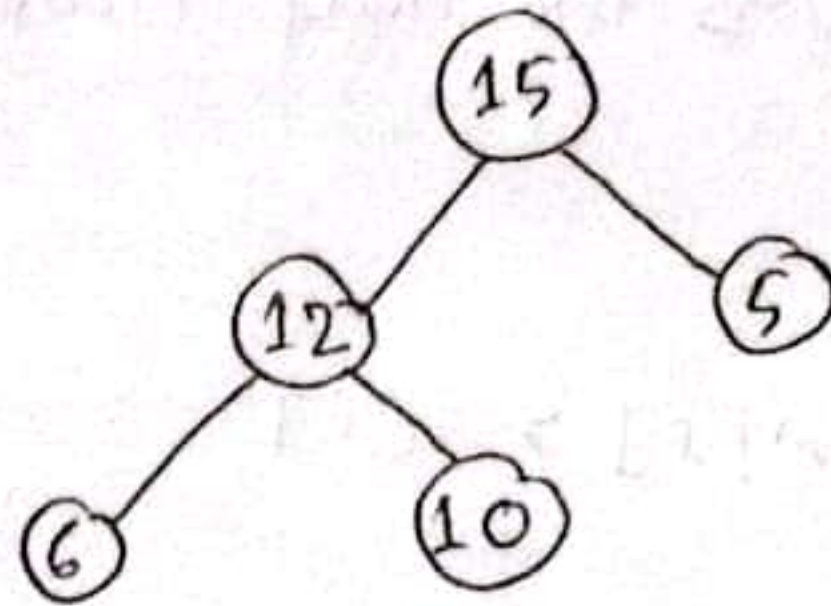
$$T(n) = an^2 + bn + c \\ = O(n^2)$$

iii) The worst-case and best-case running times are $O(n^2)$ and $\Omega(n^2)$ respectively.

HEAP

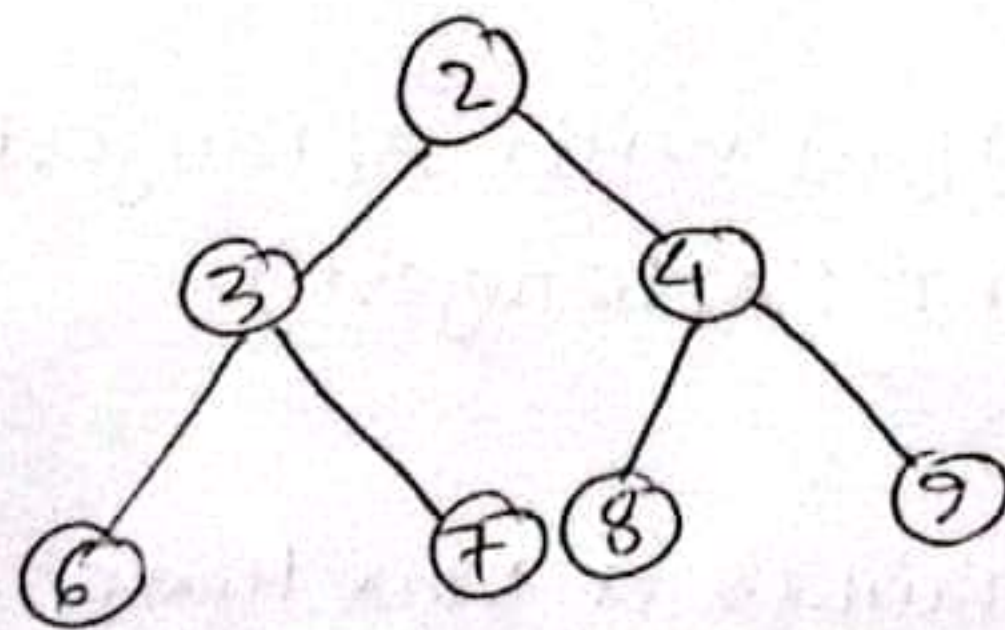
Max-Heap

The root contains the ~~maximum~~ largest element of the tree or sub-tree.



Min-Heap

The root contains the lowest element of the tree or sub-tree.



- ▣ If there are total n elements in a heap, the height of the heap, $h = \lfloor \log_2 n \rfloor$
- ▣ If " h " is the height of a heap, the maximum number of nodes $= 2^{h+1} - 1$
- ▣ In an n -element heap, there are $\lceil \frac{n}{2} \rceil$ leaf nodes and $n - \lceil \frac{n}{2} \rceil$ are non-leaf nodes.

Number of nodes at height "h" is at most

$$\left\lceil \frac{n}{2^{h+1}} \right\rceil$$

Related Algorithms

MAX-HEAPIFY (A, i) // Maintains the heap property

1. $l = 2i$
2. $r = 2i + 1$
3. if $l \leq A.\text{heap-size}$ and $A[l] > A[i]$
 $\text{largest} = l$
- 4.
5. else
 $\text{largest} = i$
- 6.
7. if $r \leq A.\text{heap-size}$ and $A[r] > A[\text{largest}]$
 $\text{largest} = r$
- 8.
9. if $\text{largest} \neq i$
 exchange $A[i]$ with $A[\text{largest}]$
10. $\text{MAX-HEAPIFY}(A, \text{largest})$
- 11.

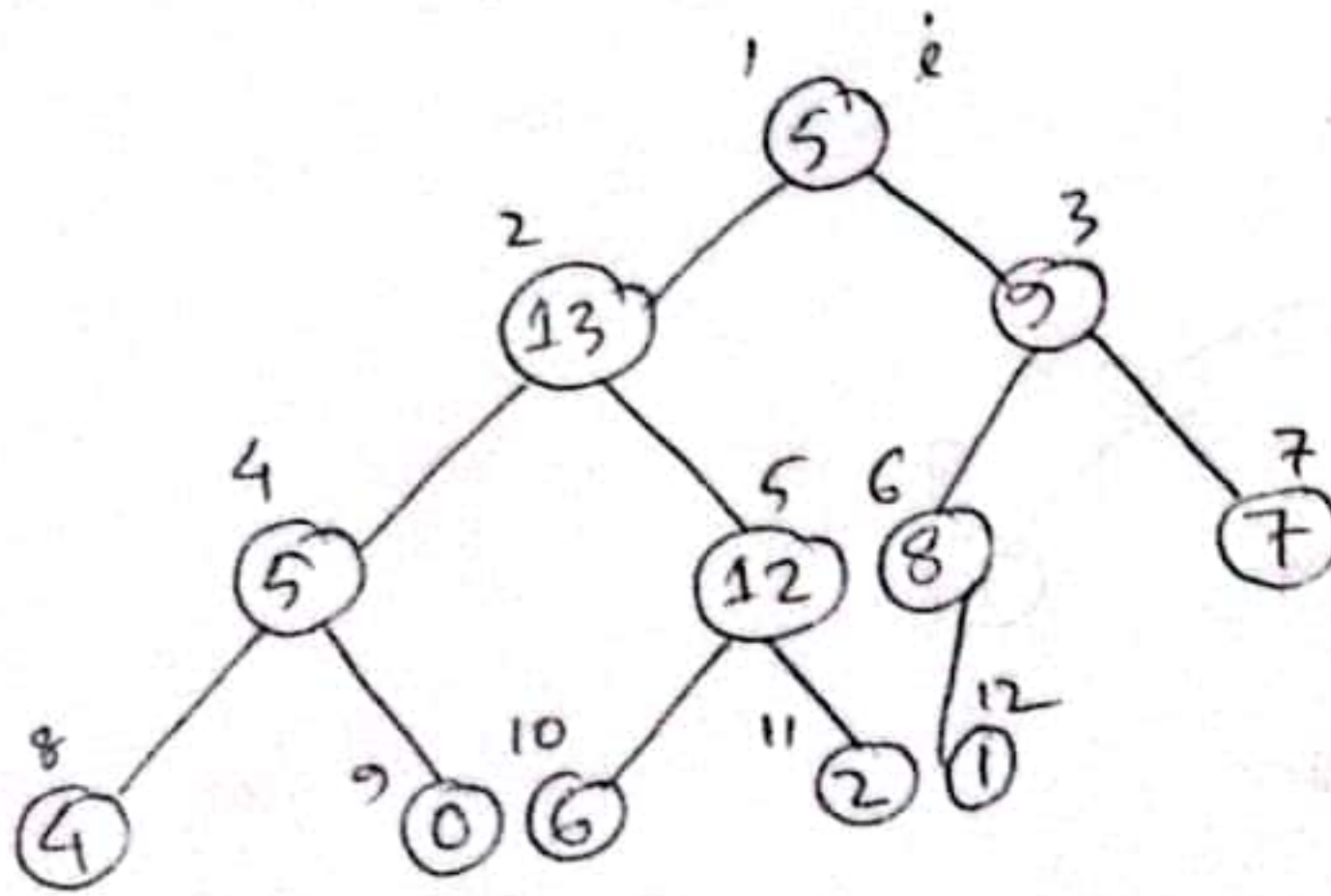
BUILD-MAX-HEAP (A) // Builds a Max Heap

1. $A.\text{heap-size} = A.\text{length}$
2. for $i = \lfloor A.\text{length} / 2 \rfloor$ down to 1
 $\text{MAX-HEAPIFY}(A, i)$
- 3.

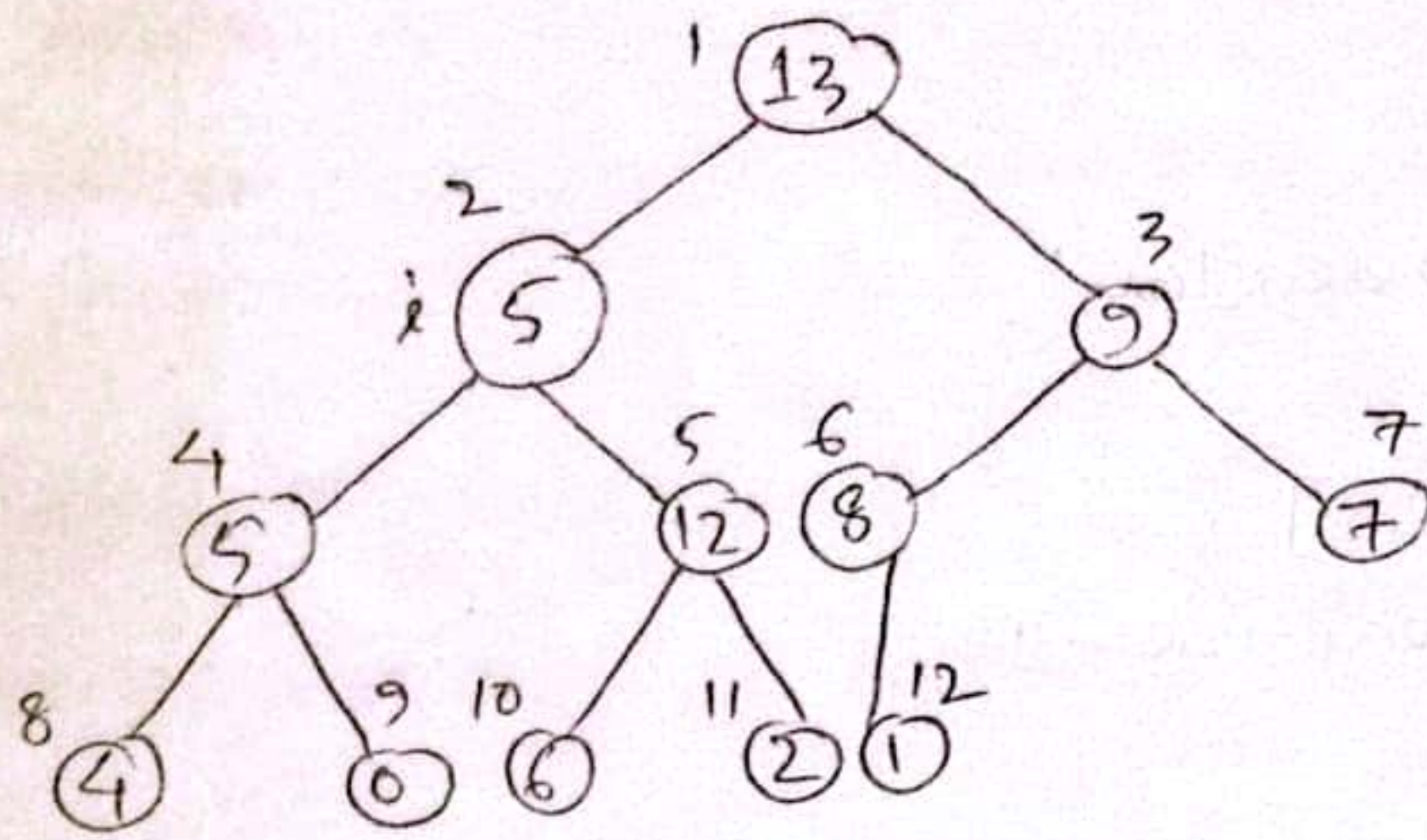
HEAP-EXTRACT-MAX(A)

1. if $A.\text{heap-size} < 1$
2. error "heap underflow"
3. $\text{max} = A[1]$
4. $A[1] = A[A.\text{heap-size}]$
5. $A.\text{heap-size} = A.\text{heap-size} - 1$
6. MAX-HEAPIFY(A, 1)
7. return max

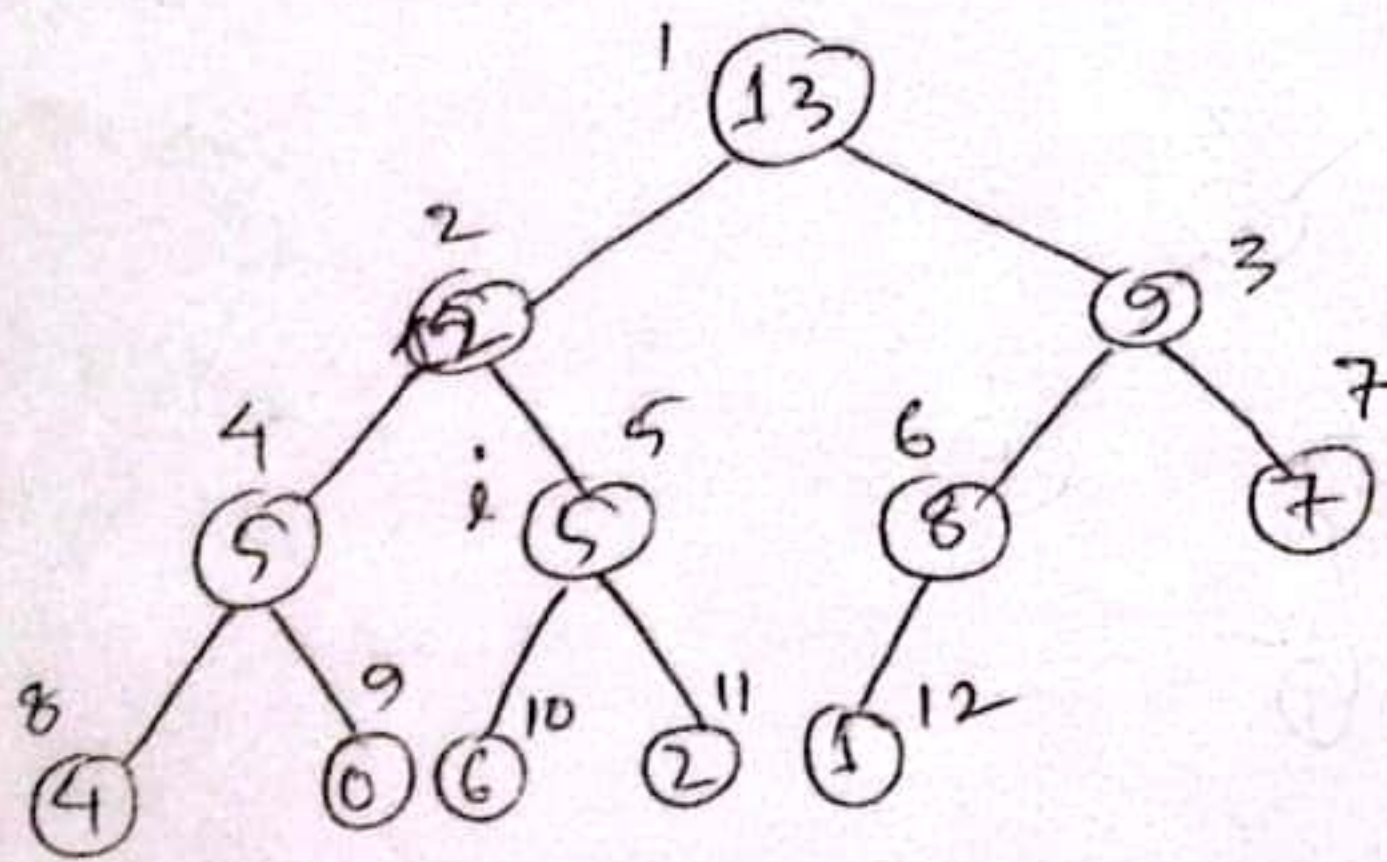
▣ Illustrate the operation of MAX-HEAPIFY(A, 1) on the following array: $A = \langle 5, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1 \rangle$



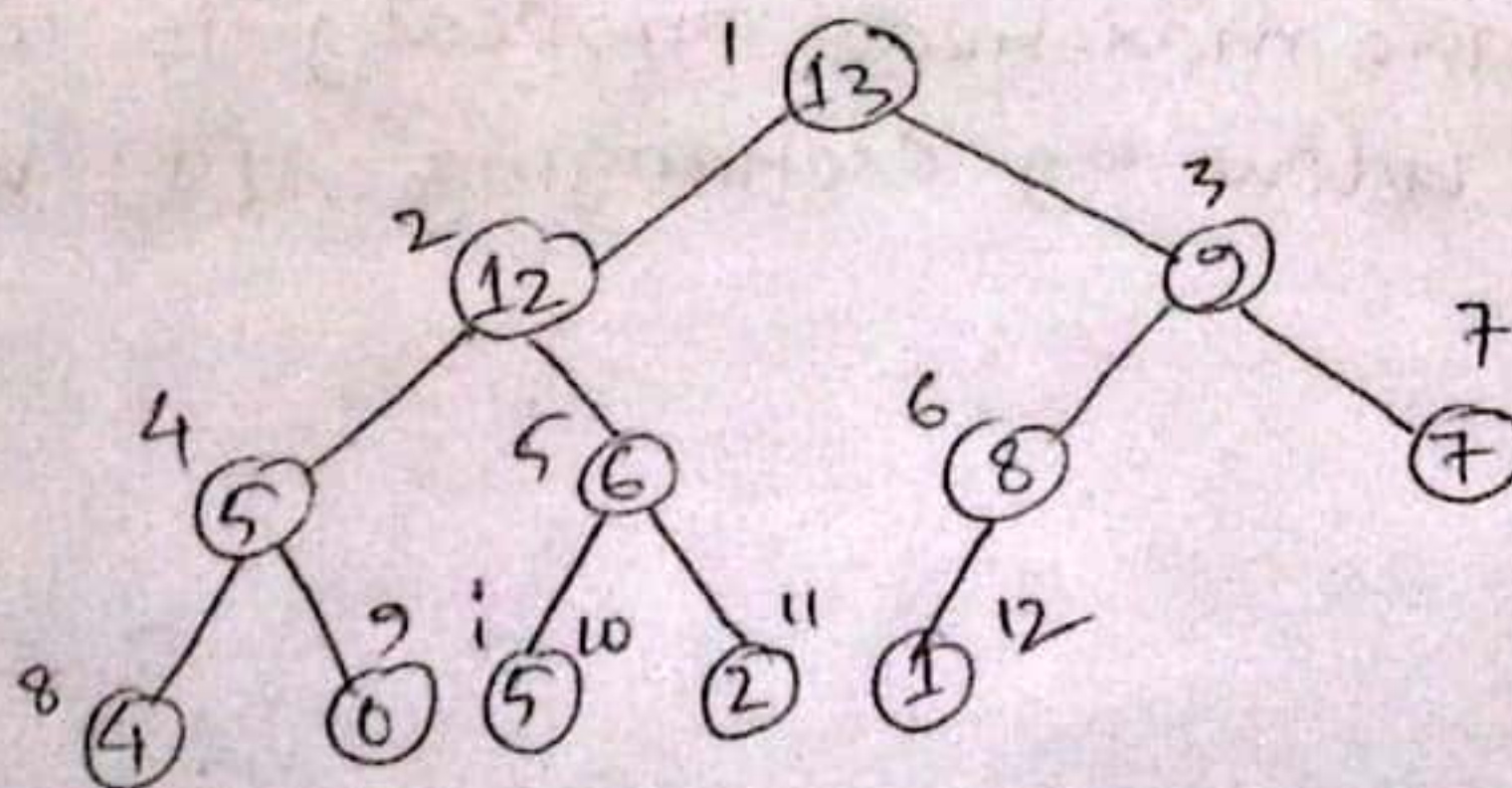
The initial configuration, with $A[1]$ at node $i = 1$ is violating the max-heap property since it is not greater than both children. The max-heap property is restored for node 1 in figure below by exchanging $A[1]$ with $A[2]$.



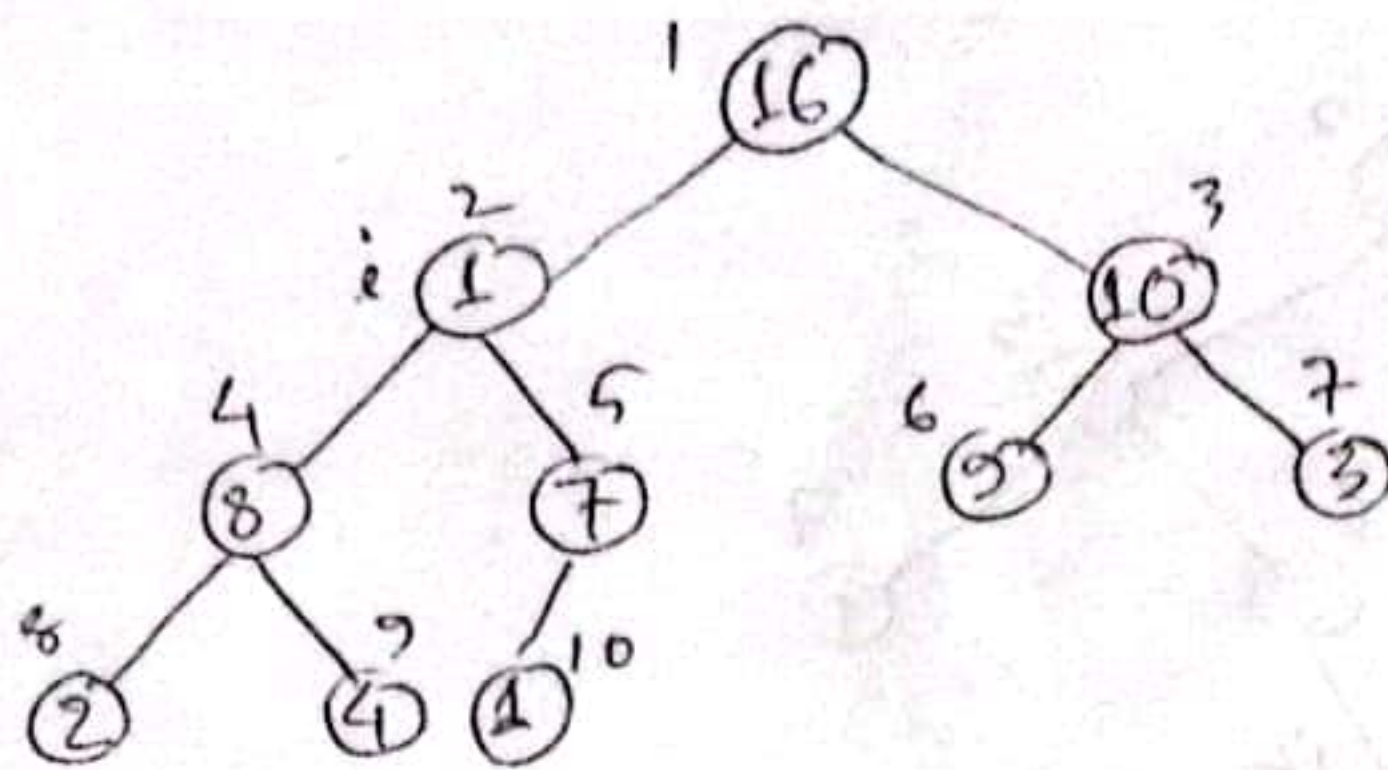
Now this configuration violates max-heap property for node 2. The recursive call MAX-HEAPIFY (A, 2) now has $i = 2$. To restore the property, A[2] and A[5] will be swapped.



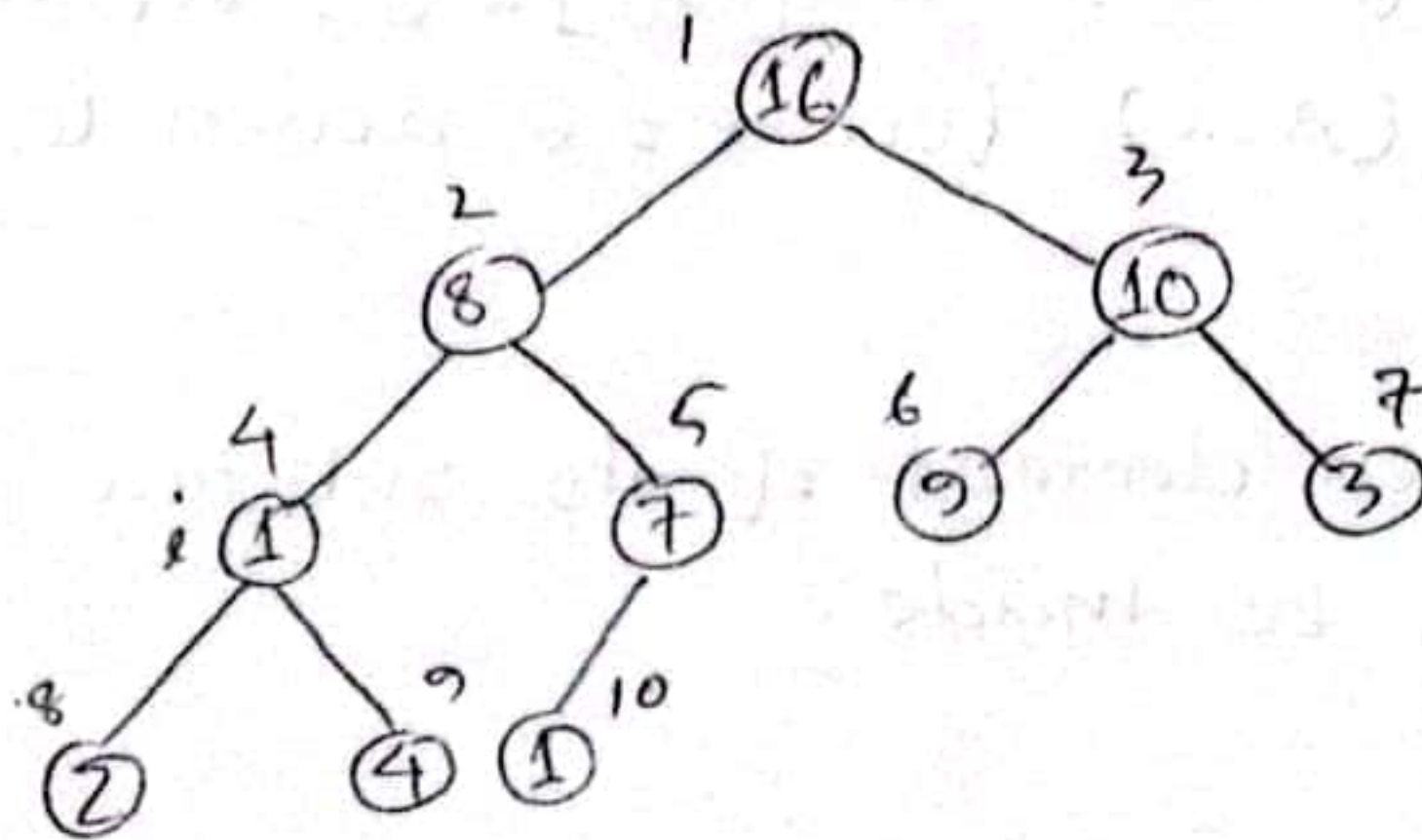
Max-heap property is violated again for node $i = 5$. So, A[5] will be swapped with A[10].



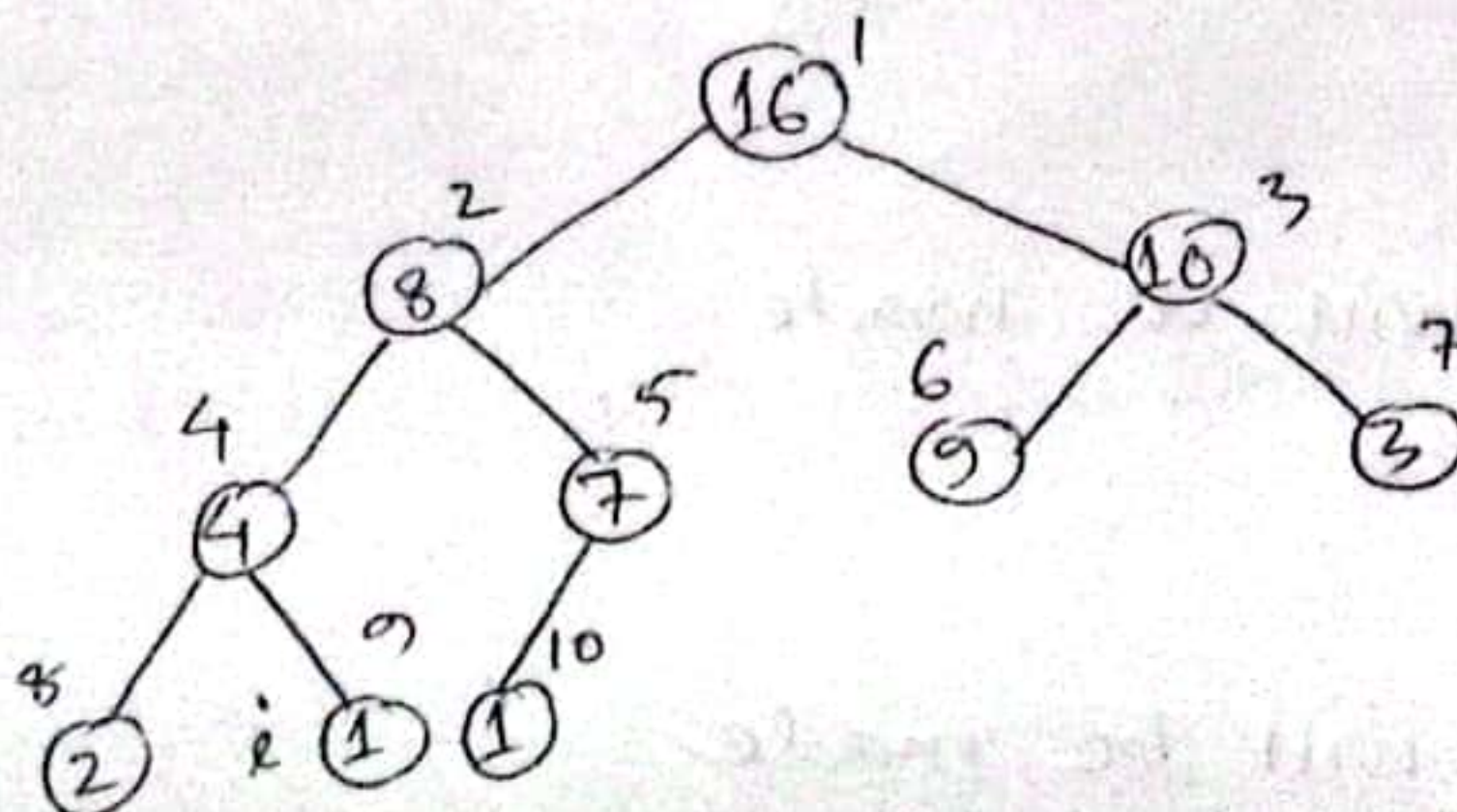
Illustrate the operation MAX-HEAPIFY(A, 2) on the array : $A = \langle 16, 1, 10, 8, 7, 9, 3, 2, 4, 1 \rangle$



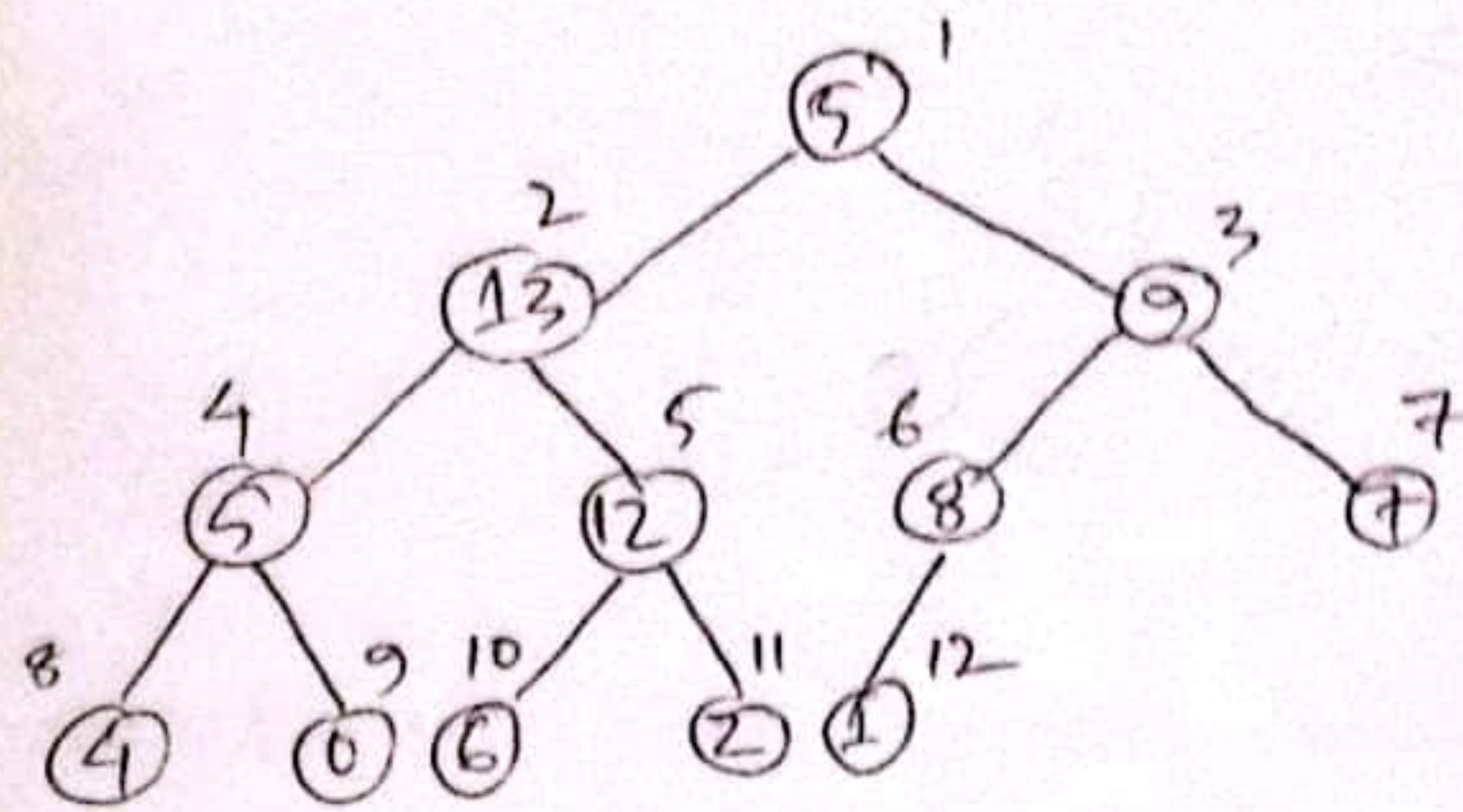
$A[2]$ violates the max-heap property, will be exchanged with largest child $A[4]$.



Again $A[4]$ violates the max-heap property, will be exchanged with largest child $A[9]$.



Q1 Illustrate the Build-Max-Heap(A) on the following array: $A = \langle 5, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1 \rangle$



Here, $A.\text{heap-size} = A.\text{length} = 12$. The non-leaf nodes starts from 1 to $\lfloor 12/2 \rfloor = 6$. We will call $\text{MAX-HEAPIFY}(A, i)$ for $i = 6$ down to 1.

$i = 6$
→

$A[6]$ is the largest element of its sub-tree.
No changes will be made.

$i = 5$
→

$A[5]$ is the largest element of its sub-tree.
No changes will be made.

$i = 4$
→

No changes will be made.

$i = 3$
→

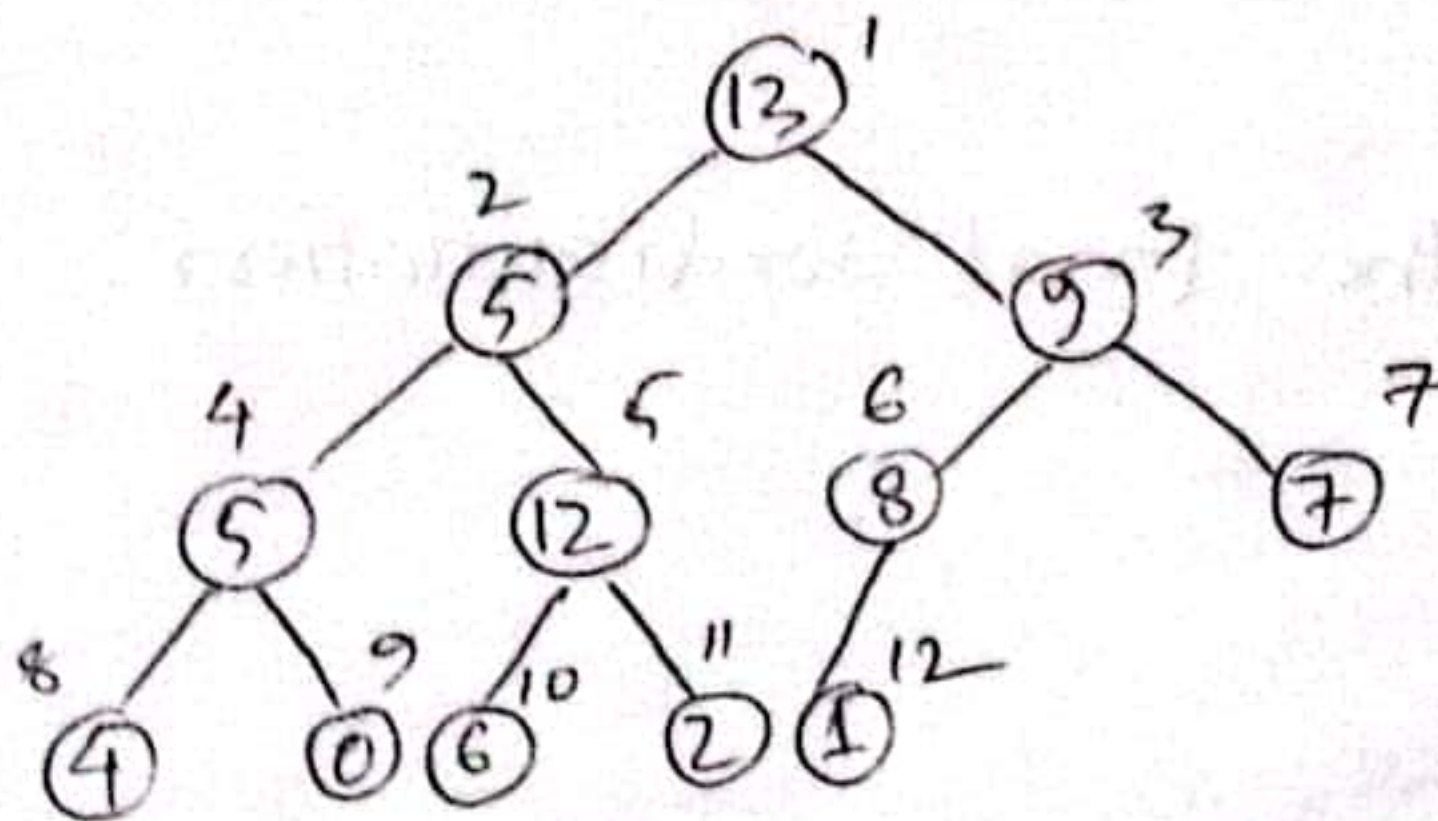
No changes will be made.

$i=2$ →

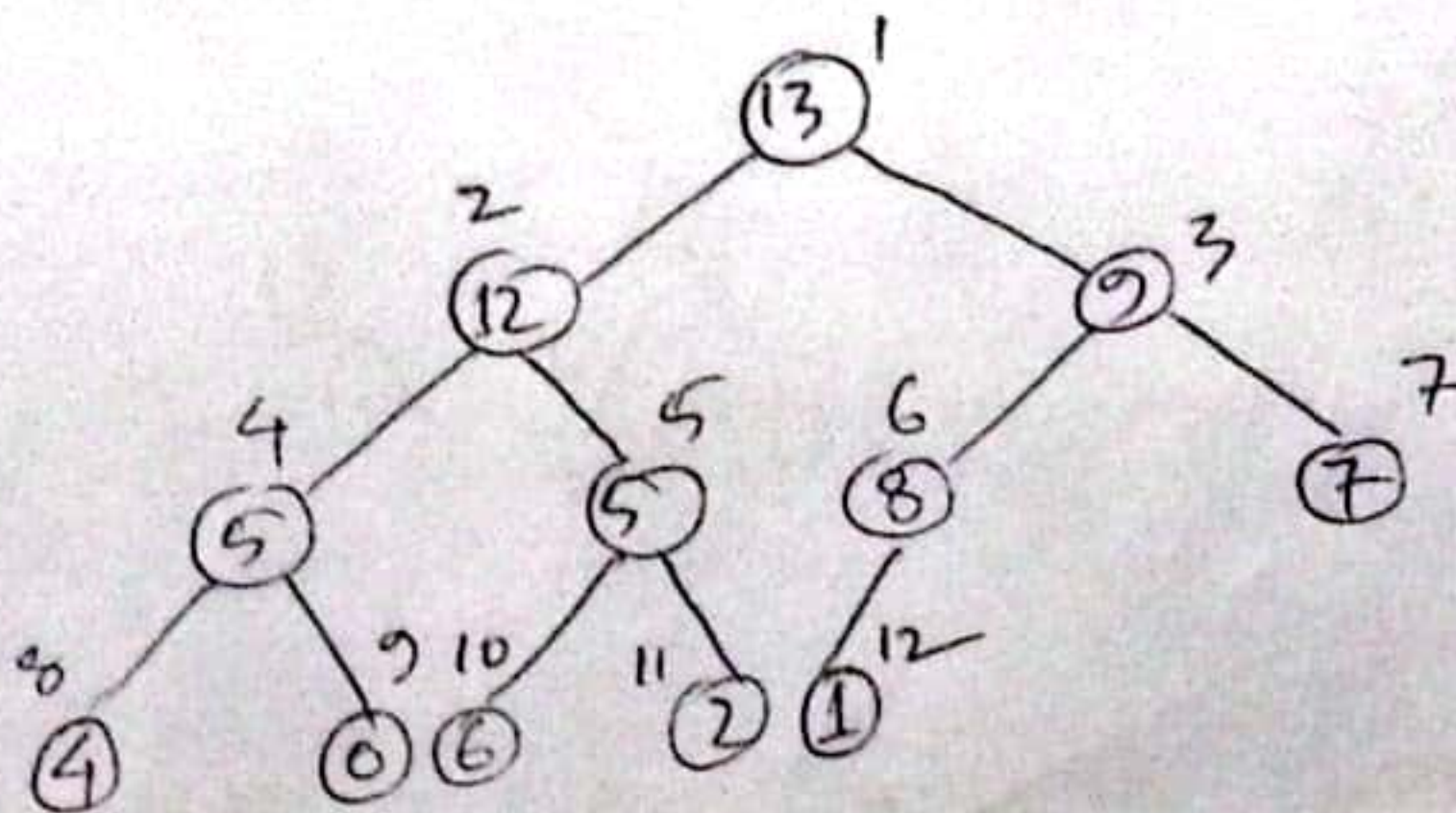
No changes will be made.

$i=1$ →

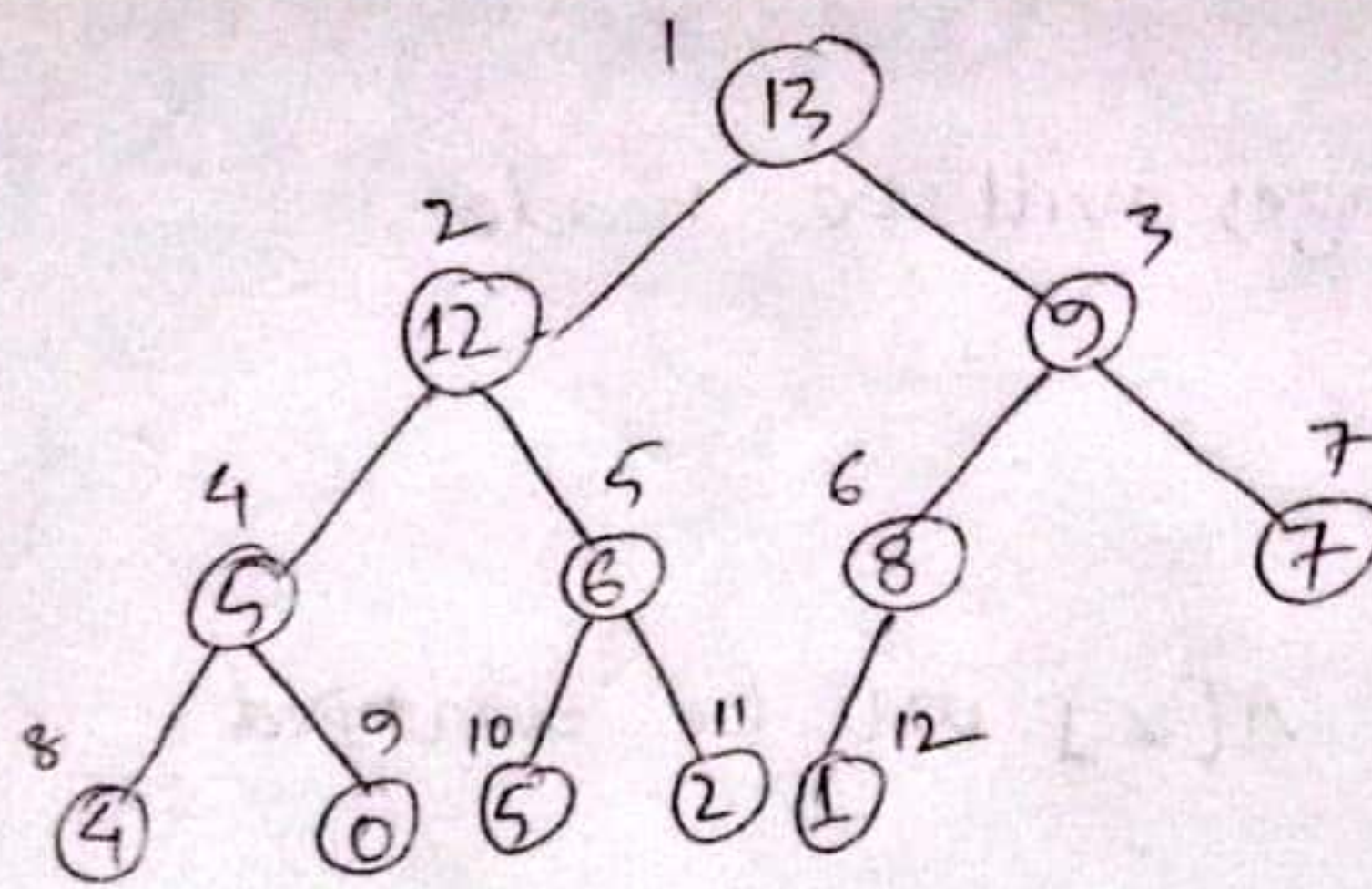
$A[1]$ and $A[2]$ will be swapped.



$A[2]$ and $A[5]$ will be swapped since $A[2]$ violates the max-heap property.

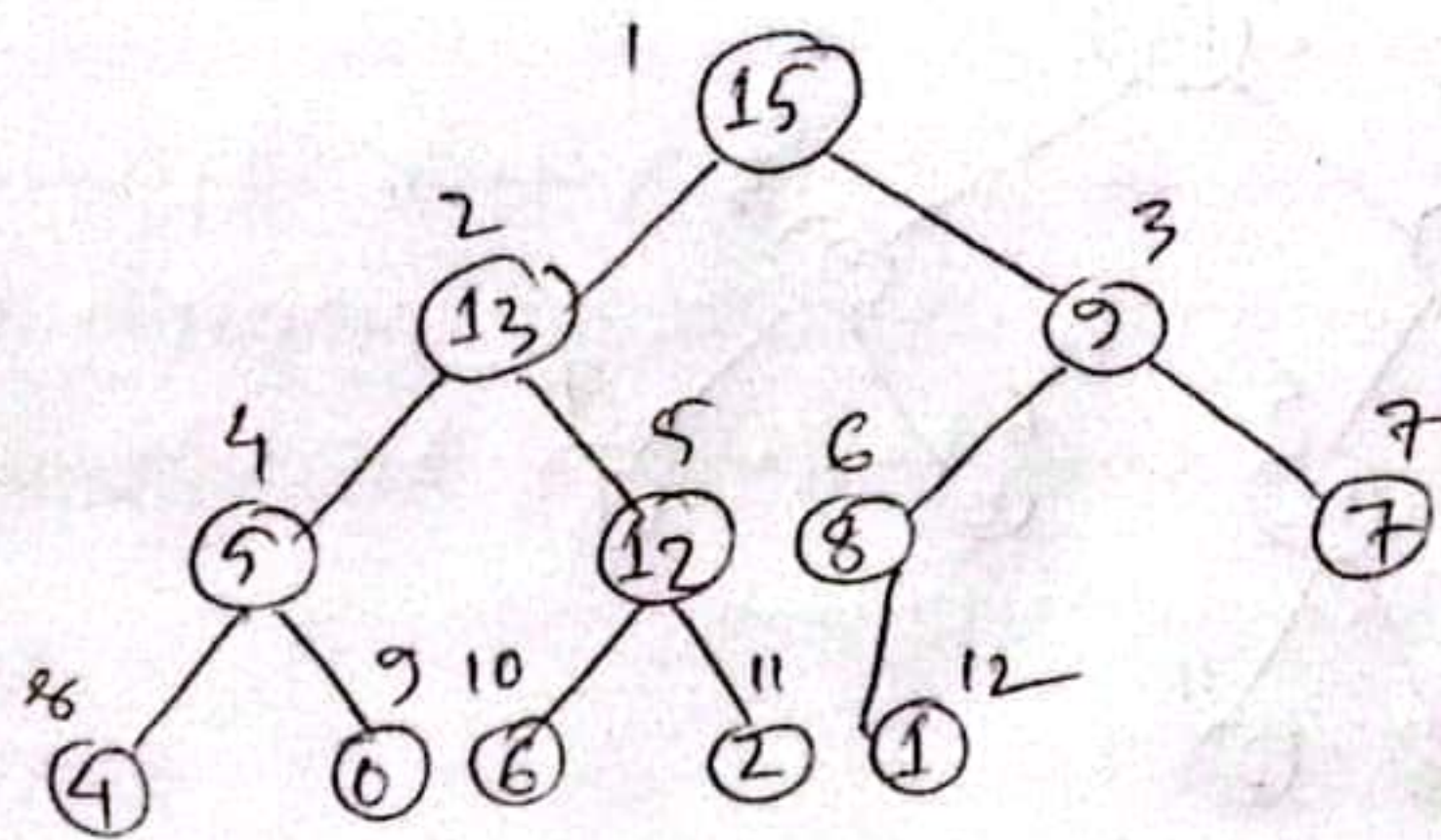


$A[5]$ and $A[10]$ will be swapped.



This is the final configuration .

Q Illustrate the operation of HEAP-EXTRACT-MAX on the heap
 $A = \langle 15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1 \rangle$

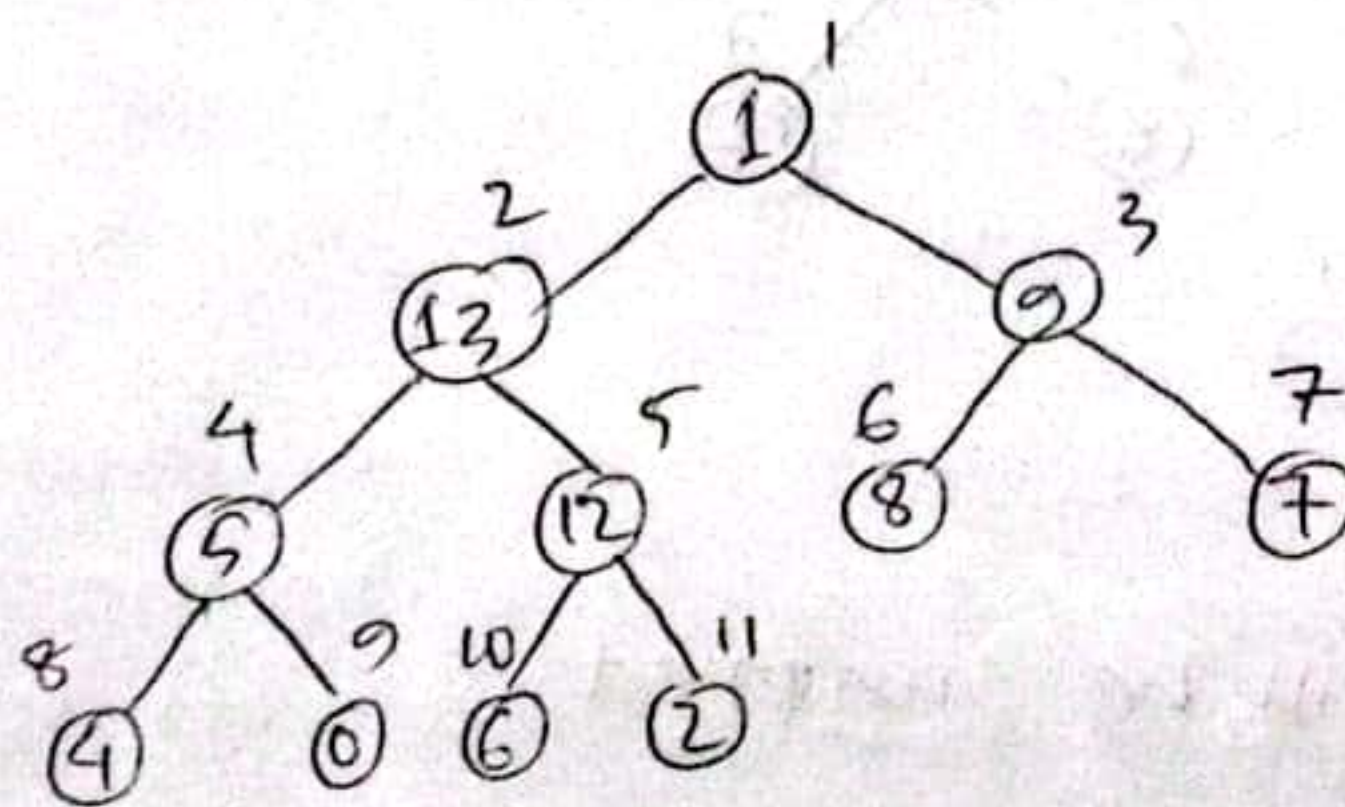


Here, $A.\text{heap-size} = 12$

$\text{max} = A[1] = 15$

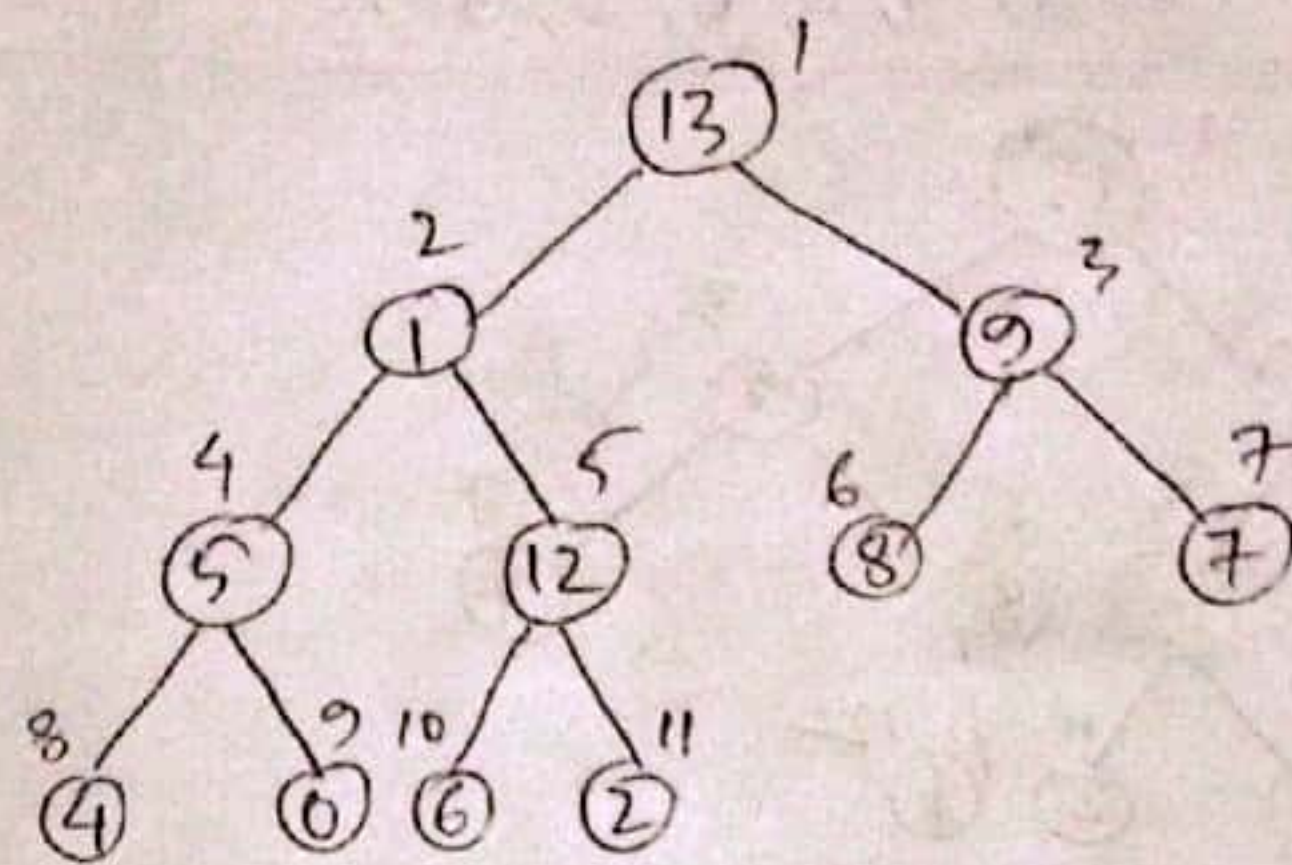
replaced by $A[12]$

Now, $A[1]$ ~~max/A[12]~~ will be swapped and
 $A.\text{heap-size}$ will become $12 - 1 = 11$.

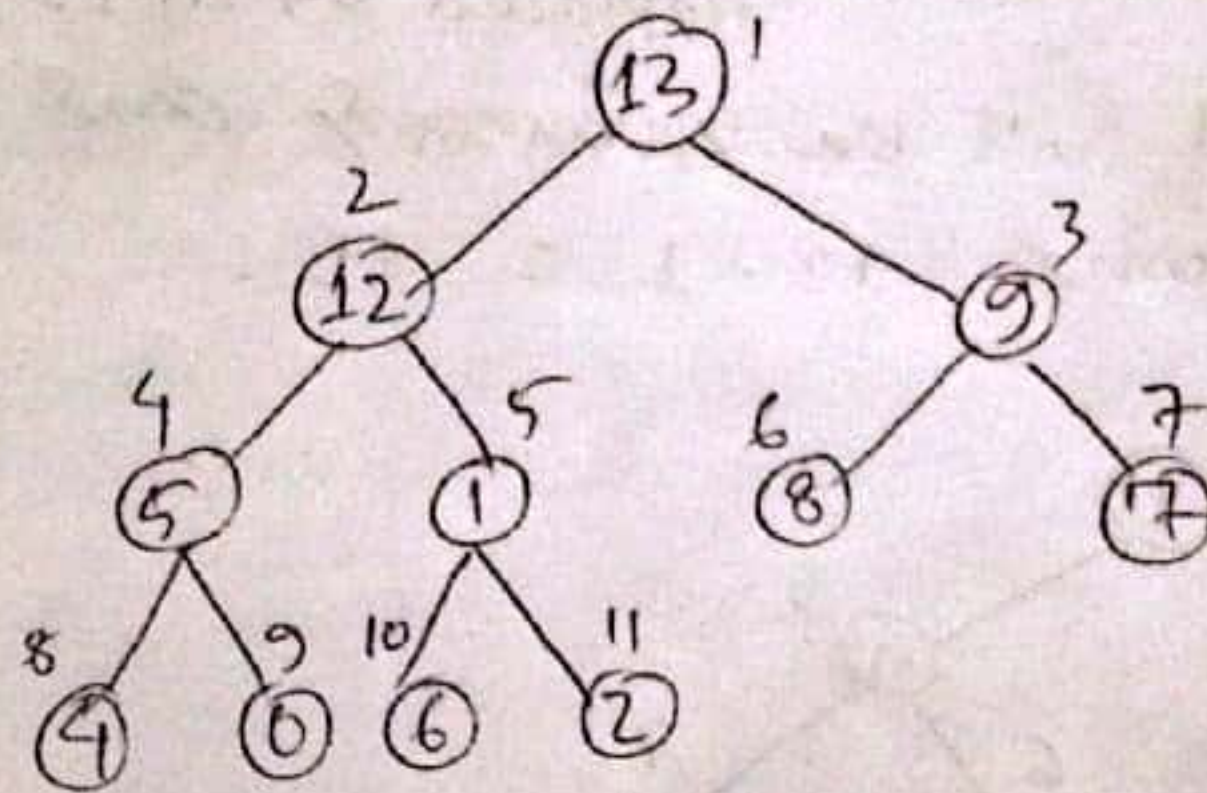


Now, $\text{MAX-HEAPIFY}(A, 1)$ will be called.

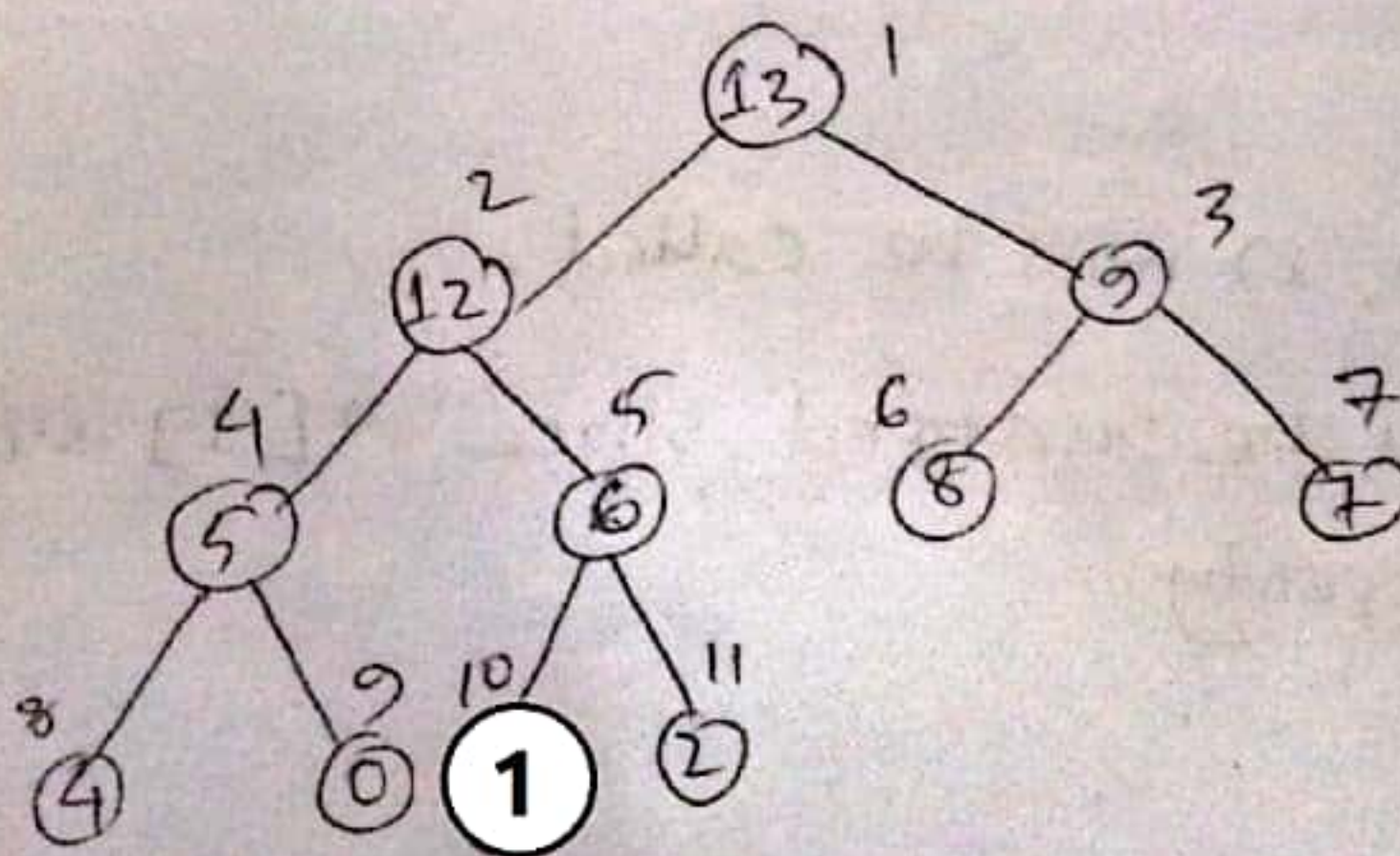
$A[2]$ and $A[1]$ will be swapped since $A[1]$ violates
the max-heap property.



$A[2]$ and $A[5]$ will be swapped.



$A[5]$ and $A[10]$ will be swapped.

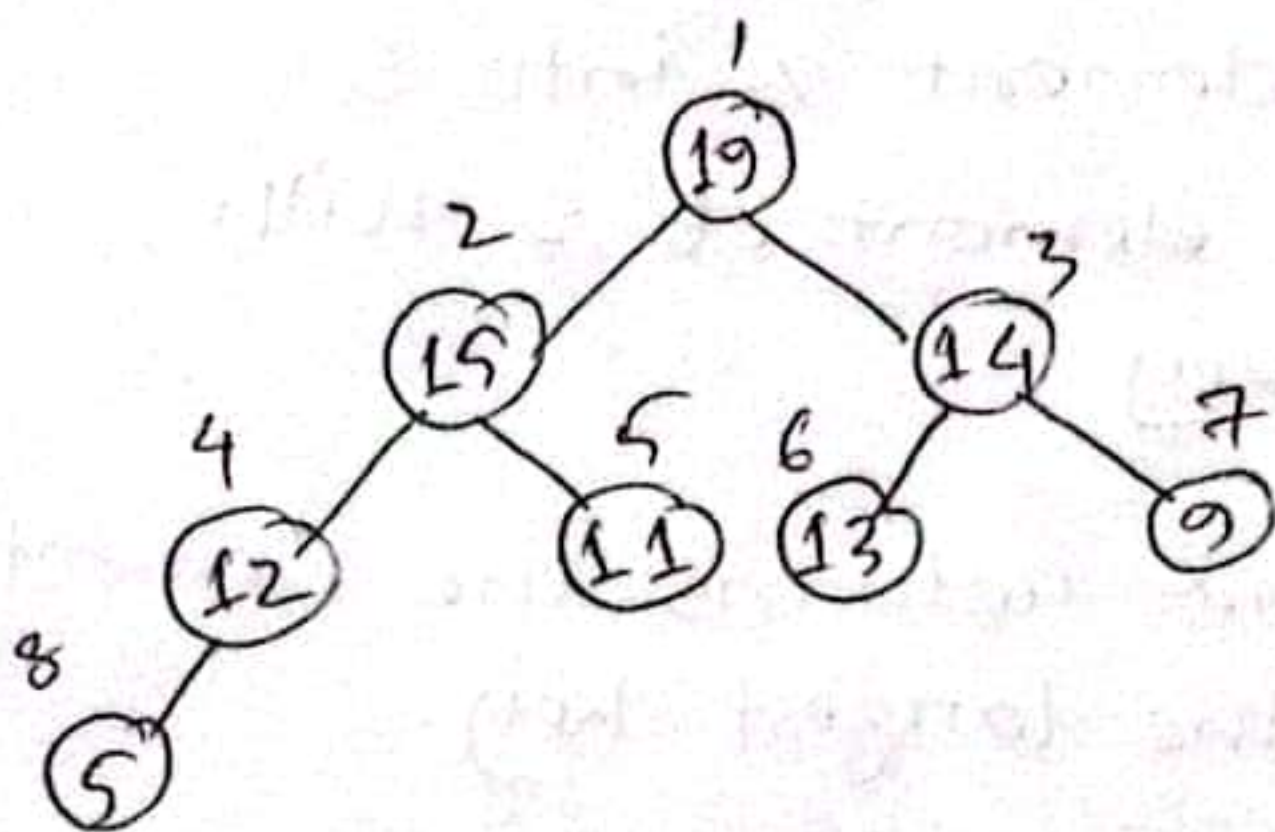


This is the final configuration of the given heap.
15 will be returned.

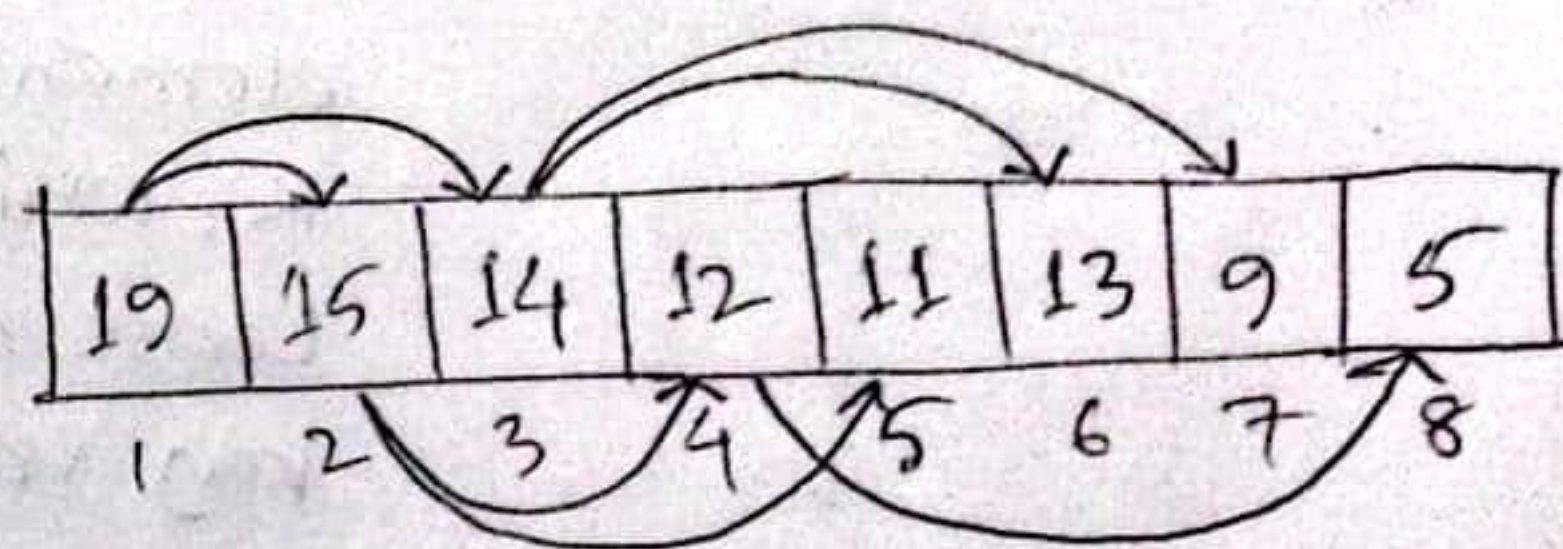
Q Suppose we have an array A where all the elements are same. Can it be considered as a Max-Heap? Can it be considered as a Min-Heap?

The given array A can be considered as both a Max-Heap and a Min-Heap.

Q With suitable example, show how a max-heap can be viewed as i) A binary tree ii) An array



(a)



(b)

Fig: (a) Tree view of Max-heap
(b) Array view of Max-heap

Q. What are the operations supported by a priority queue?

A priority queue is a data structure for maintaining a set S of elements, each with an associated value called a key. A priority queue can be a 1) Max-Priority-Queue or 2) Min-Priority-Queue.

A max priority queue supports the following operations:

- 1) $\text{INSERT}(S, x)$: inserts the element x into S
- 2) $\text{MAXIMUM}(S)$: returns the element of S with the largest key
- 3) $\text{EXTRACT-MAX}(S)$: removes and returns the element of S with the largest key.
- 4) $\text{INCREASE-KEY}(S, x, k)$: increases the value of element x 's key to the new value k , which is assumed to be at least as large as x 's current key value.

Apart from $\text{INSERT}(S, x)$ operation, a min-priority queue supports $\text{MINIMUM}(S)$, $\text{EXTRACT-MIN}(S)$, $\text{DECREASE-KEY}(S, x, k)$.

MASTER THEOREM

$$T(n) = aT(n/b) + O(n^k \log^p n)$$

$a \geq 1$, $b > 1$, $k \geq 0$ and p is a real number.

1) If $a > b^k$, then $T(n) = O(n^{\log_b a})$

2) If $a = b^k$,

i) If $p > -1$, then $T(n) = O(n^{\log_b a} \log^{p+1} n)$

ii) If $p = -1$, then $T(n) = O(n^{\log_b a} \log \log n)$

iii) If $p < -1$, then $T(n) = O(n^{\log_b a})$

3) If $a < b^k$,

i) If $p \geq 0$, then $T(n) = O(n^k \log^p n)$

ii) If $p < 0$, then $T(n) = O(n^k)$

Q Solve the recurrences using ~~master~~ master method / theorem :

i) $T(n) = 4T(n/2) + n^3$

ii) $T(n) = 5T(n/5) + n^2$

iii) $T(n) = 9T(n/3) + n$

iv) $T(n) = 2^n T(n/2) + n^n$

v) $T(n) = 2T(n/2) + n \log n$

vi) $T(n) = 2T(n/2) + \frac{n}{\log n}$

vii) $T(n) = 0.5T(n/2) + \frac{1}{n}$

viii) $T(n) = 54T(n/8) - n^2 \log n$

ix) $T(n) = 2T(n/2) + \sqrt{n}$

x) $T(n) = 4T(n/2) + cn$

i) Given,

$$T(n) = 4T(n/2) + n^3$$

Here,

$$a = 4,$$

$$b = 2,$$

$$k = 3,$$

$$p = 0$$

$$b^k = 2^3 = 8, \therefore a < b^k.$$

$$\text{Since } p > 0, T(n) = \Theta(n^k \log^p n)$$

$$= \Theta(n^3 \log^0 n)$$

$$= \Theta(n^3)$$

ii) Given,

$$T(n) = 5T(n/5) + n^2$$

Here,

$$a = 5,$$

$$b = 5,$$

$$k = 2,$$

$$p = 0$$

$$b^k = 5^2 = 25, \therefore a < b^k.$$

$$\begin{aligned}\text{Since } p > 0, \quad T(n) &= \Theta(n^k \log^p n) \\ &= \Theta(n^2 \log^0 n) \\ &= \Theta(n^2)\end{aligned}$$

iii)

Given,

$$T(n) = 9T(n/3) + n$$

Here,

$$a = 9,$$

$$b = 3,$$

$$k = 1,$$

$$p = 0$$

$$b^k = 3^1 = 3, \therefore a > b^k$$

$$\therefore T(n) = \Theta(n^{\log_b a})$$

$$= \Theta(n^{\log_3 9})$$

$$= \Theta(n^{\log_3 3^2})$$

$$= \Theta(n^{2 \log_3 3})$$

$$= \Theta(n^2)$$

iv) Given,

$$T(n) = 2^n T(n/2) + n^n$$

Comparing the given relation with,

$$T(n) = aT(n/b) + \theta(n^k \log^p n), \text{ we get}$$

$a = 2^n$. But a must be any constant where $a \geq 1$. So, we can't apply Master Theorem to solve the relation.

v) Given,

$$T(n) = 2T(n/2) + n \log n$$

Here,

$$a = 2,$$

$$b = 2,$$

$$k = 1,$$

$$p = 1$$

$$b^k = 2^1 = 2, \therefore a = b^k$$

$$\text{Since } p \geq -1, T(n) = \theta(n^{\log_b a} \log^{p+1} n)$$

$$= \theta(n^{\log_2 2} \log^{1+1} n)$$

$$= \theta(n \log^2 n)$$

$$= \theta(n \log \log n)$$

i) Given,

$$T(n) = 2T(n/2) + \frac{n}{\log n}$$

$$= 2T(n/2) + n \log^{-1} n$$

Here,

$$a = 2,$$

$$b = 2,$$

$$k = 1,$$

$$P = -1$$

$$b^k = 2^1 = 2, \therefore a = b^k.$$

$$\begin{aligned} \text{Since } P = -1, T(n) &= O(n^{\log_b a} \log \log n) \\ &= O(n^{\log_2 2} \log \log n) \\ &= O(n \log \log n) \end{aligned}$$

ii)

Given,

$$T(n) = 0.5T(n/2) + \frac{1}{n}$$

Since $a = 0.5$, i.e. $a < 1$, Master Theorem can't be applied here.

viii)

Given,

$$T(n) = 54T(n/8) - n^2 \log n$$

Master Theorem can't be applied here due to negative term i.e. $-n^2 \log n$.

ix) Given,

$$T(n) = 2T(n/2) + \sqrt{n}$$

Here,

$$a = 2,$$

$$b = 2,$$

$$k = 1/2,$$

$$p = 0$$

$$b^k = 2^{1/2}, \therefore a > b^k$$

$$\therefore T(n) = \theta(n^{\log_b a})$$

$$= \theta(n^{\log_2 2})$$

$$= \theta(n)$$

x) Given,

$$T(n) = 4T(n/2) + cn$$

Here,

$$a = 4,$$

$$b = 2,$$

$$k = 1,$$

$$p = 0.$$

$$b^k = 2^1 = 2, \therefore a > b^k$$

$$\text{So, } T(n) = \theta(n^{\log_b a})$$

$$= \theta(n^{\log_2 4})$$

$$= \theta(n^{\log_2 2^2})$$

$$= \theta(n^{2 \log_2 2})$$

$$= \theta(n^2)$$

xii) Given,

$$T(n) = 4T(n/16) + \sqrt{n}$$

Here,

$$a = 4,$$

$$b = 16,$$

$$k = 1/2,$$

$$p = 0$$

$$b^k = 16^{1/2} = 4, \therefore a = b^k.$$

$$\begin{aligned} \text{Since } p = 0 \text{ i.e. } p > -1, T(n) &= \Theta(n^{\log_b a} \log^{p+1} n) \\ &= \Theta(n^{\log_{16} 4} \log^{0+1} n) \\ &= \Theta(n^{\log_{16} \sqrt{16}} \log n) \\ &= \Theta(n^{\log_{16} 16^{1/2}} \log n) \\ &= \Theta(n^{1/2 \log_{16} 16} \log n) \\ &= \Theta(\sqrt{n} \log n) \end{aligned}$$